

**BELAJAR MUDAH
LARAVEL 9
LEVEL PEMULA**

UU No 28 tahun 2014 tentang Hak Cipta

Fungsi dan sifat hak cipta Pasal 4

Hak Cipta sebagaimana dimaksud dalam Pasal 3 huruf a merupakan hak eksklusif yang terdiri atas hak moral dan hak ekonomi.

Pembatasan Pelindungan Pasal 26

Ketentuan sebagaimana dimaksud dalam Pasal 23, Pasal 24, dan Pasal 25 tidak berlaku terhadap:

- i. penggunaan kutipan singkat Ciptaan dan/atau produk Hak Terkait untuk pelaporan peristiwa aktual yang ditujukan hanya untuk keperluan penyediaan *informasi* aktual;
- ii. Penggandaan Ciptaan dan/atau produk Hak Terkait hanya untuk kepentingan penelitian ilmu pengetahuan;
- iii. Penggandaan Ciptaan dan/atau produk Hak Terkait hanya untuk keperluan pengajaran, kecuali pertunjukan dan Fonogram yang telah dilakukan Pengumuman sebagai bahan ajar; dan
- iv. penggunaan untuk kepentingan pendidikan dan pengembangan ilmu pengetahuan yang memungkinkan suatu Ciptaan dan/atau produk Hak Terkait dapat digunakan tanpa izin Pelaku Pertunjukan, Produser Fonogram, atau Lembaga Penyiaran.

Sanksi Pelanggaran Pasal 113

1. Setiap Orang yang dengan tanpa hak melakukan pelanggaran hak ekonomi sebagaimana dimaksud dalam Pasal 9 ayat (1) huruf i untuk Penggunaan Secara Komersial dipidana dengan pidana penjara paling lama 1 (satu) tahun dan/atau pidana denda paling banyak Rp100.000.000 (seratus juta rupiah).
2. Setiap Orang yang dengan tanpa hak dan/atau tanpa izin Pencipta atau pemegang Hak Cipta melakukan pelanggaran hak ekonomi Pencipta sebagaimana dimaksud dalam Pasal 9 ayat (1) huruf c, huruf d, huruf f, dan/atau huruf h untuk Penggunaan Secara Komersial dipidana dengan pidana penjara paling lama 3 (tiga) tahun dan/atau pidana denda paling banyak Rp500.000.000,00 (lima ratus juta rupiah).

BELAJAR MUDAH LARAVEL 9 LEVEL PEMULA



Raja Sabaruddin, M.Kom.

Sri Murni, M.Kom.

Lisnawanty, S.T., M.Kom.

Wahyu Nugraha, M.Kom.



**PT Insan Cendekia
Mandiri Group**

Belajar Mudah Laravel 9 Level Pemula

**Raja Sabaruddin, M.Kom., Sri Murni, M.Kom., Lisnawanty, S.T., M.Kom.,
dan Wahyu Nugraha, M.Kom.**

Editor:

Lailatul Marhamah

Desainer:

Nur Aziza

Sumber Gambar Kover:

www.canva.com

Penata Letak:

Lailatul Marhamah

Proofreader:

Tim ICM

Ukuran:

x, 182 hlm, 14,8x21 cm

ISBN:

978-623-179-534-2

Cetakan Pertama:

Januari 2024

Hak cipta dilindungi undang-undang
Dilarang keras menerjemahkan, memfotokopi, atau
memperbanyak sebagian atau seluruh isi buku ini
tanpa izin tertulis dari Penerbit.

Anggota IKAPI : 020/SBA/20

**PENERBIT INSAN CENDEKIA MANDIRI
(PT. INSAN CENDEKIA MANDIRI GROUP)**

Jorong Pale, Nagari Pematang Panjang, Kecamatan Sijunjung,
Kabupaten Sijunjung, Provinsi Sumatra Barat – Indonesia 27554
HP/WA: 0813-7272-5118

Website: www.insancendekiamandiri.co.id

E-mail: insancendekiamandirigroup@gmail.com



Daftar Isi

Prakata | ix

BAB 1 DEFINISI LARAVEL | 1

- A. *Faramework* | 1
- B. *PHP Framework* | 2
- C. *PHP Framework Vs PHP Native* | 3
- D. *Why Framework?* | 3
- E. *Apa itu Laravel?* | 6
- F. *Manfaat Laravel Apa Sih?* | 7

BAB 2 INSTAL LARAVEL | 9

- A. *Persiapan Lingkungan* | 9
- B. *Instal Laravel* | 9
- C. *Mengakses Laravel* | 11

BAB 3 MENGENAL ROUTE & VIEW | 17

- A. *Apa itu Route?* | 17
- B. *Apa itu View?* | 17
- C. *Mengenal Route & View* | 17
- D. *Membuat Route & View* | 20
- E. *Latihan* | 22



BAB 4 MENGENAL VARIABEL PHP DI LARAVEL | 24

BAB 5 CONTROLLER PADA LARAVEL | 29

- A. Apa itu Controller? | 29
- B. Cara Membuat Controller di Laravel | 29
- C. Cara Menggunakan Controller di Laravel | 34
- D. Latihan | 37

BAB 6 *PASSING DATA* CONTROLLER KE VIEW | 39

- A. *Passing Data* dari Controller ke View | 39
- B. *Passing Data* Array | 42

BAB 7 REQUEST DATA DARI URL | 47

- A. Request Data dari URL | 47
- B. Request Data dari Inputan | 49

BAB 8 KONFIGURASI DATABASE DI MySQL | 55

- A. Konfigurasi Dasar pada Laravel | 55
- B. *Caching* Konfigurasi | 58
- C. *Debug Mode* | 59
- D. *Maintenance Mode* | 59

BAB 9 SIMPLE TEMPLATE DI LARAVEL | 63

- A. Membuat Sistem *Blade Simple Template* | 63
- B. Latihan | 72



BAB 10 FUNGSI MATEMATIKA DI LARAVEL | 73

- A. Bilangan Berpangkat: Pow() | 76
- B. Akar Bilangan: sqrt() | 80
- C. Latihan | 83

BAB 11 FUNGSI *FORM* PADA LARAVEL | 85

- A. Pengenalan *Form* | 85
- B. Komponen *Form* | 86
- C. Pengolahan Data *Form* | 86
- D. Membuat Inputan di Laravel | 87
- E. Latihan | 94

BAB 12 STUDI KASUS SEDERHANA | 95

- A. Pembelian Tiket | 95
- B. Latihan | 103

BAB 13 MEMBUAT CRUD | 105

- A. Membuat CRUD dengan Query Builder | 105
- B. Pengaturan *Database* | 105
- C. Persiapan *Database* dan Tabel Karyawan | 107
- D. Menampilkan Data dari *Database* (Read) | 107
- E. Menginput Data ke *Database* (Create) | 114
- F. *Update Data* pada *Database* (Update) | 118
- G. Menghapus Data dari *Database* (Delete) | 123
- H. Kode Lengkap Toko_Sederhana | 125
- I. Latihan | 135



BAB 14 MEMBUAT LAPORAN SEDERHANA DI LARAVEL | 137

- A. Kode Program | 137
- B. Latihan | 148

BAB 15 MEMBUAT PAGINATION DI LARAVEL | 149

- A. Definisi Pagination | 149
- B. Membuat Pagination | 149

BAB 16 PERINTAH Pencarian Di Laravel | 157

BAB 17 PENOMORAN FIELD OTOMATIS DI LARAVEL | 167

Daftar Pustaka | 175

Indeks | 177

Profil Penulis | 179





Prakata

Selamat datang dalam perjalanan pembelajaran pengembangan web dengan buku “*Belajar Laravel 9 Level Pemula*”. Buku ini dirancang khusus bagi para pemula yang ingin memahami secara mendalam konsep-konsep pengembangan aplikasi web menggunakan *framework* PHP Laravel.

Panduan ini dimulai dengan membahas definisi Laravel, memahami peran *framework* dalam pengembangan web, hingga perbandingan antara menggunakan *framework* dan pendekatan *native* dalam bahasa pemrograman PHP. Melalui penjelasan yang sistematis, pembaca akan diajak untuk memahami mengapa penggunaan *framework* seperti Laravel sangat dianjurkan dalam pengembangan proyek web.

Buku ini tidak hanya memberikan pengetahuan teoretis, tetapi juga memberikan panduan praktis dalam langkah-langkah instalasi Laravel dan persiapan lingkungan pengembangan. Pembaca akan dibimbing untuk mengenal konsep dasar seperti route, view, dan controller, serta bagaimana cara mengakses dan mengelola data dari berbagai sumber.

Panduan langkah demi langkah disajikan dalam membuat CRUD (*Create, Read, Update, Delete*), membuka wawasan pada konsep *form* dan pengolahan data *form* dalam Laravel, serta membahas studi kasus sederhana sebagai latihan praktis. Pembaca juga akan diajak melangkah lebih jauh dengan pembuatan laporan sederhana, implementasi *pagination*, dan pemahaman tentang pencarian dan penomoran otomatis di Laravel.



Selamat menikmati pembelajaran! Semoga buku ini menjadi panduan yang berharga dalam memahami dan menguasai pengembangan web dengan *framework* Laravel.

Salam hangat,

Penulis



BAB 1



DEFINISI LARAVEL

Laravel merupakan salah satu *framework* PHP yang saat ini banyak di minati, maka ada baiknya kita akan memahami beberapa teori terkait *framework* dan definisi Laravel

A. *Faramework*

Framework adalah kumpulan kode program dengan aturan penulisan tertentu yang bertujuan untuk memudahkan serta mempercepat pembuatan aplikasi. *Framework* pada awalnya berasal dari kebutuhan *programmer* untuk mengurangi pembuatan kode yang sama berulang kali.

Sebagai contoh, misalkan saya bekerja sebagai *programmer* di sebuah perusahaan *software*. *Project* pertama disuruh membuat sistem informasi sekolah, di mana perlu fitur pendaftaran siswa dan *login* siswa. Setelah selesai, lanjut ke *project* kedua untuk membuat aplikasi perpustakaan yang di antaranya perlu halaman pendaftaran karyawan serta *login* karyawan. Selanjutnya *project* ketiga membuat sistem informasi akademik kampus yang juga perlu fitur pendaftaran dosen dan *login* dosen.

Sampai di sini, bisa dilihat bahwa untuk ketiga *project* kita perlu membuat fitur pendaftaran dan *login*. Besar kemungkinan untuk *project* keempat dan



seterusnya juga butuh fitur serupa. Daripada membuat dari awal terus menerus, akan lebih efisien jika kita menyiapkan sebuah kode dasar untuk halaman pendaftaran dan *login*. Maka pekerjaan selesai dengan lebih cepat.

Seiring bertambahnya pengalaman, kode dasar ini kita modifikasi dengan tambahan berbagai fitur baru agar semakin lengkap, misalnya menambah cara koneksi ke *database*, validasi *form*, dan sebagainya. Akhirnya, jadilah sebuah *framework*.

B. PHP Framework

Jadi sederhananya PHP *Framework* adalah *framework* yang dibuat menggunakan bahasa pemrograman PHP. PHP *framework* dibuat sebagai sebuah **Model View Controller** (MVC), yaitu perangkat yang dapat membagi arsitektur pemrograman ke dalam bagian-bagian yang lebih spesifik, sehingga masing-masing komponen bisa dimodifikasi tanpa mengubah komponen lainnya.

Dengan MVC, perubahan fungsi dalam aplikasi tidak akan berpengaruh terhadap komponen antarmukanya. Hal ini bisa terjadi karena perangkat tersebut membagi pemrograman ke dalam tiga area yaitu **Model**, **View**, dan **Controller**.

1. **Model** mengacu pada data aplikasi
2. **View** mengacu pada komponen visual antarmuka
3. **Controller** mengacu pada fungsi atau logika aplikasi

Terdapat berbagai pilihan *framework* PHP seperti CodeIgniter, Symfony, Yii, Zend dan tentu saja Laravel.



C. PHP Framework Vs PHP Native

Selain menggunakan PHP *framework* kita juga bisa membangun aplikasi dengan PHP *Native*. Untuk mempelajari PHP *native* ini relatif lebih mudah di banding *framework*.

Native berarti membangun sendiri dari awal. Artinya kita membuat aplikasi mulai dari nol. Dengan *Native* kita bisa membuat sebuah sistem yang “*Just What We Need*”, sistem yang benar-benar dipakai khusus untuk tujuan tertentu saja.

Berikut Alasan kenapa kita tidak perlu menggunakan *framework* dan cukup dengan php *native* adalah:

1. Aplikasi yang dibuat cukup sederhana. Kurang pas jika menggunakan *framework* hanya untuk membuat program menghitung luas segitiga. Untuk yang seperti ini sebaiknya pakai PHP *native* saja karena akan jauh selesai lebih cepat.
2. Ingin mengejar *performa*. Menggunakan *Native running time* jauh lebih cepat, sehingga cocok untuk kondisi yang *Low Resource* atau menangani *Big Data*, sedangkan *Framework* tentu saja lebih boros, karena banyak dari *Resource* yang disediakan *framework* ternyata tidak kita gunakan.

D. Why Framework?

Kemudian apa alasan sebaiknya menggunakan *framework*? Tujuan utama kenapa menggunakan *framework* adalah untuk mempercepat pembuatan aplikasi, karena di dalam *framework* sudah tersedia berbagai fitur siap pakai. Kita tinggal menggunakan fitur ini tanpa perlu membuat semuanya dari nol. Selain itu



aturan penulisan di *framework* akan memaksa kita menggunakan cara penulisan yang baik. Selain mempercepat proses pembuatan aplikasi, penggunaan PHP *framework* juga memberi sejumlah manfaat, yaitu:

1. Tersedia fitur siap pakai. *Framework* sudah menyediakan berbagai komponen siap pakai untuk membantu kita dalam merancang aplikasi. Sebagai contoh, di Laravel dengan 1 perintah sederhana kita bisa meng-*generate form* register lengkap dengan fitur *login* dan *logout*. Jika menggunakan PHP *native*, membuat fitur ini bisa butuh waktu seharian penuh. Tidak hanya itu, umumnya *framework* PHP menyediakan cara singkat untuk membuat fitur-fitur lain, seperti pembuatan *form*, validasi, menampilkan pesan error, mengakses *database*, pembuatan *layout*, dsb.
2. Mengikuti *best practice*. *Framework* memiliki aturan penulisan baku yang tidak bisa diubah. Dengan demikian kita akan “dipaksa” mengikuti cara penulisan *framework*. Cara penulisan ini sudah dipikirkan oleh ribuan *programmer* profesional yang merancang *framework* tersebut. Sebagai contoh, sebagian besar *framework* PHP menggunakan konsep MVC, yakni singkatan dari Model, View dan Controller. Konsep MVC bertujuan untuk memisahkan 3 bagian program: kode untuk mengakses *database* (disebut sebagai model), kode untuk tampilan (*view*) dan kode untuk mengatur alur logika program (*controller*). Dengan pemisahan ini, aplikasi kita menjadi lebih rapi dan mudah dikelola.
3. Mudah untuk kolaborasi. Menggunakan *framework* juga memudahkan pembuatan kode program yang



dibuat oleh tim. *Framework* memiliki aturan penulisan yang sudah baku sehingga setiap anggota tim bisa dengan mudah membaca alur kode program yang dibuat oleh *programmer* lain. Misalnya jika ada masalah dengan *database*, maka problem utama kemungkinan besar ada di model (bagian 'M' dari MVC). Selain itu, konsep MVC juga membuat pemisahan antara *front-end* dan *back-end* yang lebih baku. Tim *front-end* yang mengembangkan *web design* bisa fokus ke sisi tampilan saja (**view**), sedangkan tim *back-end* bisa fokus ke alur logika aplikasi (**controller** dan **model**).

4. Mudah membaca kode program. Konsep yang sudah baku juga bermanfaat, jika ada pergantian anggota tim. Tanpa *framework*, butuh waktu lama bagi *programmer* baru untuk memahami kode program yang sudah ada. Sangat mungkin konsep berpikir *programmer* sebelumnya berbeda jauh dengan *programmer* yang akan menggantikan. Namun, jika aplikasi tersebut dibuat dengan *framework*, setiap *programmer* harus mengikuti cara yang sudah diatur oleh *framework* tersebut. Misalnya kode program untuk pengaturan tampilan ada di **View**, pengaturan *database* ada di **Model**, dsb. sehingga lebih mudah bagi *programmer* baru untuk melakukan modifikasi.
5. Keamanan Aplikasi. Sebagai contoh, ada celah kelemahan web yang dikenal sebagai CSRF atau *Cross-Site Request Forgery*, yakni teknik mengisi *form* (seperti *form login*), dengan kode program yang bukan berasal dari *website* kita. Celah ini bisa dimanfaatkan untuk membuat bot atau program yang terus-menerus mencoba mengisi *form login*. Mayoritas *framework*



PHP (termasuk Laravel), sudah menyediakan cara untuk mengatasi masalah ini, yaitu dengan memaksa kita untuk membuat sebuah *CSRF token*. Laravel akan menampilkan error jika sebuah *form* tidak memiliki *CSRF token*. Cara input *token* ini juga sangat sederhana, hanya perlu satu perintah singkat. Fitur keamanan seperti ini sudah lebih dahulu dipikirkan oleh tim pengembang Laravel, sehingga aplikasi yang kita buat relatif lebih aman.

E. Apa itu Laravel?

Sekarang kita akan bahas Pengertian Laravel. Laravel adalah *framework* web PHP gratis *open source* yang dikembangkan oleh Taylor Otwell. Laravel ditujukan untuk pengembangan aplikasi berbasis web, khususnya bagi mereka yang mengikuti pola arsitektur MVC (Model View Controller). Saat ini Laravel menjadi salah satu *framework* PHP yang paling banyak digunakan di dunia. Karena kemudahan penggunaannya, banyaknya dukungan seperti dokumentasi yang baik dan dukungan library yang lengkap, sehingga bisa kita sebut jika “Laravel seperti memanjakan penggunanya”.

Laravel juga menggunakan metode MVC (Model, View, Controller). Apa itu MVC? MVC merupakan suatu metode untuk memisahkan bagian-bagian web aplikasi. Yang disebut dengan MODEL biasanya bagian yang berurusan dengan *database*. VIEW adalah bagian antarmuka atau bagian depan aplikasi, segala sesuatu yang diproses dalam sistem akan ditampilkan pada **view**, sedangkan **controller** adalah bagian yang menangani atau penghubung antar **model** dan **view**, jadi **controller** lah yang berperan sebagai pengendali dari sebuah sistem.



F. Manfaat Laravel Apa Sih?

Kepopuleran Laravel juga bisa dilihat dari kebutuhan industri. Jika Anda mencari lowongan kerja *programmer*, maka besar kemungkinan lowongan tersebut mensyaratkan harus menguasai PHP hingga *framework* Laravel. Ini juga salah satu alasan untuk belajar Laravel. Laravel menawarkan beberapa keuntungan apabila kita menggunakan Laravel sebagai *framework* dasar dalam mengembangkan *website*.

1. Pembuatan aplikasi menjadi lebih cepat.
2. Developer lebih mudah untuk membuat perencanaan, pembuatan aplikasi dan juga *maintenance*-nya.
3. Aplikasi yang dibuat lebih stabil serta andal karena *Framework* Laravel sudah teruji stabilitas dan keandalannya.
4. Developer lebih mudah membaca kode program, sehingga *bugs* lebih cepat ditemukan.
5. Lebih aman karena Laravel mengantisipasi celah keamanan yang mungkin timbul.
6. Membuat dokumentasi aplikasi lebih mudah.
7. Mudah dan Dokumentasi (tutorial) Lengkap.





BAB 2



INSTAL LARAVEL

A. Persiapan Lingkungan

Pada tahap instalasi Laravel, yaitu menggunakan *Composer*. Namun sebelum lanjut instalasi perlu persiapan lingkungan, yaitu

1. XAMPP Terbaru
<https://www.apachefriends.org/index.html>
2. *Composer* Terbaru
<https://getcomposer.org/Composer-Setup.exe>
3. *Text Editor* (di sini kami menggunakan Visual Studio Code) <https://code.visualstudio.com/download>

Setelah *download* 3 aplikasi di atas, maka silakan instal di laptop masing-masing, dengan urutan instal XAMPP terlebih dahulu kemudian dilanjutkan dengan *composer* dan *text editor*.

B. Install Laravel

Pada tahap ini kita akan menginstal Laravel. Dalam dokumentasi resminya Laravel, dijelaskan bahwa terdapat 2 cara instalasi Laravel. Pertama dengan perintah **composer create-project**, dan kedua dengan **composer global require laravel/installer**. kita akan membahas instal Laravel dengan **composer global require**



laravel/installer. Silakan buka **cmd** lalu pindah ke folder **htdocs xampp** dengan perintah berikut.

cd C:\xampp\htdocs

```
C:\> Command Prompt
Microsoft Windows [Version 10.0.19043.2006]
(c) Microsoft Corporation. All rights reserved.

C:\Users\bangr>cd C:\xampp\htdocs
```

Kemudian ketikan kode berikut.

composer global require Laravel/installer

```
C:\> Command Prompt
Microsoft Windows [Version 10.0.19043.2006]
(c) Microsoft Corporation. All rights reserved.

C:\Users\bangr>cd C:\xampp\htdocs

C:\xampp\htdocs>composer global require laravel/installer_
```

Kemudian tambahkan kode berikut

Laravel new kelas-Laravel



```
Command Prompt - laravel new kelas-laravel
C:\xampp\htdocs>composer global require laravel/installer
Changed current directory to C:/Users/bang/AppData/Local/Composer
Info from https://repo.packagist.org: #StandWithUkraine
Using version ^4.2 for laravel/installer
./composer.json has been updated
Running composer update laravel/installer
Loading composer repositories with package information
Updating dependencies
Lock file operations: 0 installs, 1 update, 0 removals
  - Upgrading laravel/installer (v4.2.16 => v4.2.17)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 0 installs, 1 update, 0 removals
  - Downloading laravel/installer (v4.2.17)
  - Upgrading laravel/installer (v4.2.16 => v4.2.17): Extracting archive
Generating autoload files
0 packages you are using are looking for funding.
Use the composer fund command to find out more!
No security vulnerability advisories found

C:\xampp\htdocs>laravel new kelas-laravel

Laravel
```

Apabila sudah tampil teks **Application key set successfully** dan kursor kembali ke **C:\xampp\ htdocs**, artinya proses instalasi Laravel sudah selesai. Silakan buka folder **htdocs** dari Windows Explorer, akan terlihat folder **coba1** yang berisi berbagai *file* Laravel.

Selain cara di atas bisa juga menggunakan kode berikut.

composer create-project laravel/Laravel latihan --prefer-dist

Kode berikut jika kita menginstal Laravel tidak bisa menggunakan cara pertama di atas. **--prefer-dist** berfungsi menyesuaikan instalasi Laravel berdasarkan PHP yang ada di laptop masing-masing. Sedangkan Latihan adalah nama *project* yang akan kita buat

C. Mengakses Laravel

Selama ini, cara kita mengakses sebuah *file* PHP adalah dengan menjalankan XAMPP lalu membukanya dari alamat *localhost*. Namun pada Laravel kita menggunakan



CMD untuk menjalankan *project* yang kita buat di folder **htdocs**.

Silakan buka **cmd** lalu masuk ke folder instalasi Laravel:

cd kelas-Laravel

```
Command Prompt

[INFO] No publishable resources for tag [laravel-assets].

No security vulnerability advisories found
> @php artisan key:generate --ansi

[INFO] Application key set successfully.

[INFO] Application ready! Build something amazing.

C:\xampp\htdocs>cd kelas-laravel_
```

Setelah berada di dalam folder Laravel (contoh *project*: kelas-Laravel), jalankan web server dengan perintah berikut lalu **enter**:

php artisan serve

```
C:\xampp\htdocs>cd kelas-laravel

C:\xampp\htdocs\kelas-laravel>php artisan serve

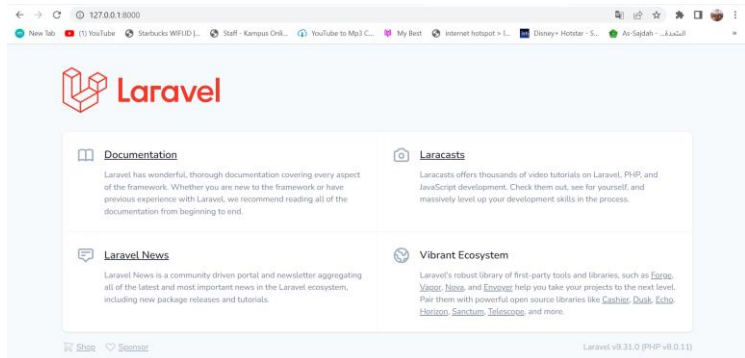
[INFO] Server running on [http://127.0.0.1:8000].

Press Ctrl+C to stop the server
```

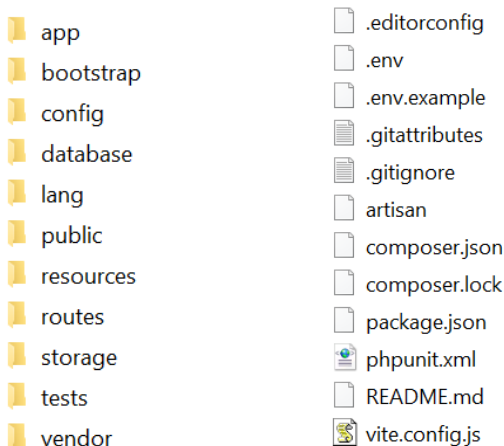
Jika tampil teks *Server running on [http://127.0.0.1:8000]*, artinya server sudah aktif dan



bisa diakses, silakan buka web browser dan ketik alamat **http://127.0.0.1:8000**



Baik, kita akan lihat apa saja folder yang ada di Laravel ini.



Berikut penjelasan singkat dari isi setiap folder Laravel:

1. **App**: berisi berbagai *file* untuk aplikasi yang akan dibangun. Dalam folder inilah nantinya kita membuat **controller** dan **model**.



2. **Bootstrap**: berisi *file* **app.php** yang berfungsi sebagai bootstrap atau *file* pengaturan awal dari Laravel. Selain itu terdapat juga folder *cache* untuk meningkatkan performa aplikasi. Sebagai info tambahan, nama folder ini tidak ada kaitannya dengan *framework* CSS *Bootstrap*. Dalam istilah komputer, *bootstrap* memiliki makna yang sama dengan *booting* yakni proses awal dari sebuah aplikasi.
3. **Config**: berisi berbagai *file* konfigurasi Laravel. Folder ini cukup sering kita akses, karena di sinilah berbagai pengaturan Laravel tersimpan.
4. **Database**: berisi folder dan *file* yang ‘mengurusi’ *database* seperti *migrations*, *factories* dan *seeds*. Folder *database* ini juga bisa dipakai sebagai tempat menyimpan *file* tabel *database* SQLite (jika Anda menggunakan SQLite).
5. **Public**: berisi *file* **index.php** sebagai *file* awal dari semua *request* ke aplikasi Laravel. *File* **index.php** inilah yang nanti memanggil berbagai *file-file* lain. Namun kita tidak akan mengedit *file* **index.php** ini secara langsung.
6. **Resource**: berisi *file* **resource** ‘mentah’ seperti *file* CSS dan JavaScript. Di sini juga nantinya kita membuat *view*.
7. **Routes**: berisi *file* yang akan menangani proses *routing*.
8. **Storage**: berisi tempat penyimpanan *file* yang di *generate* oleh Laraval, seperti *file* log. Jika kita membuat *form* **upload**, *file* tersebut juga bisa disimpan di sini.
9. **Test**: berisi *file* untuk proses testing seperti PHPUnit.
10. **Vendor**: Berisi berbagai *file* internal *framework* Laravel. Di sinilah semua library **dependency** Laravel



berada. Folder vendor ini mencakup 90% ukuran *framework* Laravel (33MB yang terdiri lebih dari 8000an *file*). Meskipun berisi banyak *file*, folder ini tidak perlu kita utak-atik.

Info:

Kita tidak perlu menjalankan web server Apache bawaan XAMPP, karena yang sedang berjalan saat ini adalah server internal bawaan PHP. Karena tidak perlu menjalankan Apache bawaan XAMPP, maka sebenarnya kita tidak harus menginstal Laravel di folder **htdocs**. Bisa saja Laravel diinstal ke *drive* D atau tempat lain kemudian jalankan perintah `php artisan serve` dari folder tersebut. Jendela cmd yang kita pakai menjalankan perintah `php artisan serve` tidak boleh ditutup, apabila jendela ini ditutup, server otomatis akan berhenti. Secara *default port* yang dipakai adalah 8000. Kita bisa ganti port ini dengan nomor lain dengan tambahan perintah **`-port nomor_port`**.





BAB 3



MENGENAL ROUTE & VIEW

A. Apa itu *Route*?

Route jika diartikan dalam bahasa Indonesia artinya rute atau jalur. Pada Laravel, *route* yang dimaksud adalah bagian yang mengatur rute aplikasi yang kita buat dengan Laravel. Rute yang kita buat akan menjadi sebuah URL lengkap yang dapat kita akses. Misalkan kita ingin membuat halaman seperti **localhost/blog**, maka kita harus membuat *route* blog. Dengan rute yang sudah dibuat tersebut, kita dapat membuka **view**, menjalankan **controller**, dan lain-lain pada saat *route* blog diakses.

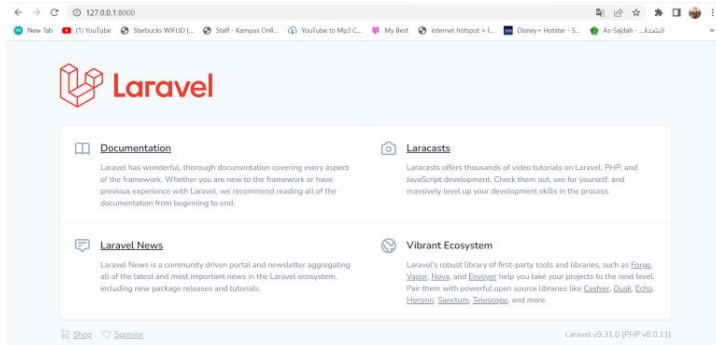
B. Apa itu *View*?

Pengertian View pada Laravel – **View** merupakan bagian khusus untuk menangani tampilan (*user interface*) untuk ditampilkan pada web browser. Hasil tampilan aplikasi yang akan kita buat nanti dinamakan **view**.

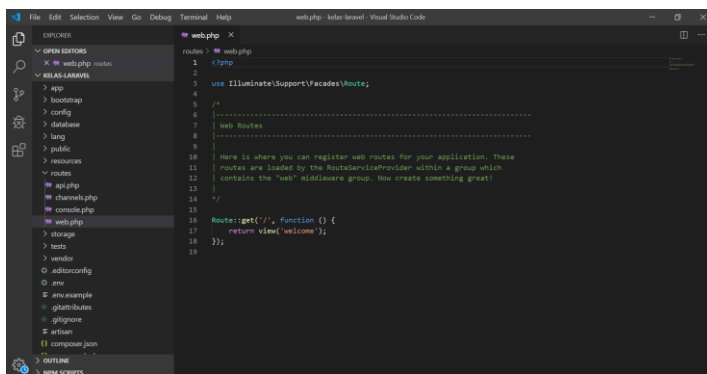
C. Mengenal *Route & View*

Setelah membaca penjelasan di atas, agar lebih paham kita langsung masuk ke contoh. Lihat tampilan awal *project* Laravel yang sudah berhasil kita instal sebelumnya.





Tampilan awal *project* Laravel ini berada pada **view** yang bernama **welcome.blade.php** letaknya di dalam folder **resources/views**. Silakan buka **file web.php** dalam folder **routes**, seperti gambar berikut:



Di dalam **file web.php** terdapat kode berikut:

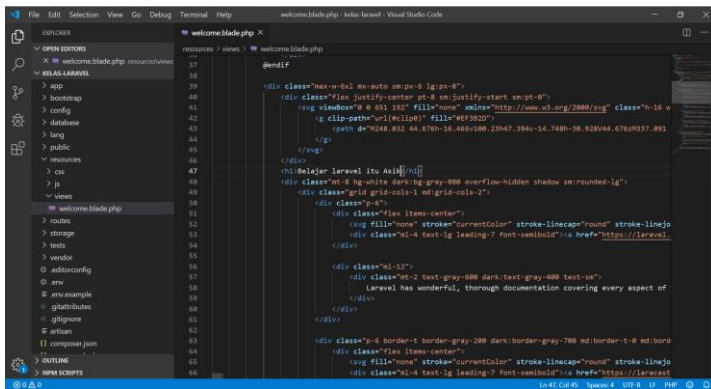
```
Route::get('/', function () {
    return view('welcome');
});
```

Maksud baris kode di atas adalah pada saat direktori awal **(/)** *project* di jalankan, maka akan menampilkan **file view welcome** untuk menjadi tampilan



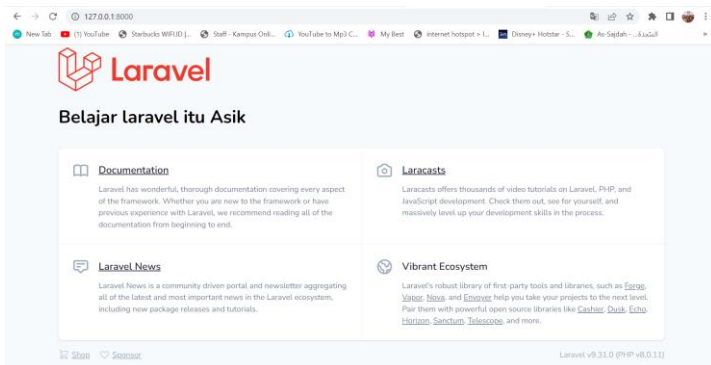
ketika alamat **http://localhost:8000** di akses pada web browser. File **welcome** tersebut berada pada folder **resources/views** dengan nama file **welcome.blade.php**.

Sekarang kita akan mengubah isi dari **welcome.blade.php**. buka file **welcome.blade.php**, coba tambahkan tulisan “**Belajar Laravel itu Asik**” di bawah logo Laravel pada baris ke 47.



```
17 <div class="flex justify-center pt-8 px:justify:space-between">
18 <div class="flex justify-center pt-8 px:justify:space-between">
19 <div class="flex justify-center pt-8 px:justify:space-between">
20 <div class="flex justify-center pt-8 px:justify:space-between">
21 <div class="flex justify-center pt-8 px:justify:space-between">
22 <div class="flex justify-center pt-8 px:justify:space-between">
23 <div class="flex justify-center pt-8 px:justify:space-between">
24 <div class="flex justify-center pt-8 px:justify:space-between">
25 <div class="flex justify-center pt-8 px:justify:space-between">
26 <div class="flex justify-center pt-8 px:justify:space-between">
27 <div class="flex justify-center pt-8 px:justify:space-between">
28 <div class="flex justify-center pt-8 px:justify:space-between">
29 <div class="flex justify-center pt-8 px:justify:space-between">
30 <div class="flex justify-center pt-8 px:justify:space-between">
31 <div class="flex justify-center pt-8 px:justify:space-between">
32 <div class="flex justify-center pt-8 px:justify:space-between">
33 <div class="flex justify-center pt-8 px:justify:space-between">
34 <div class="flex justify-center pt-8 px:justify:space-between">
35 <div class="flex justify-center pt-8 px:justify:space-between">
36 <div class="flex justify-center pt-8 px:justify:space-between">
37 <div class="flex justify-center pt-8 px:justify:space-between">
38 <div class="flex justify-center pt-8 px:justify:space-between">
39 <div class="flex justify-center pt-8 px:justify:space-between">
40 <div class="flex justify-center pt-8 px:justify:space-between">
41 <div class="flex justify-center pt-8 px:justify:space-between">
42 <div class="flex justify-center pt-8 px:justify:space-between">
43 <div class="flex justify-center pt-8 px:justify:space-between">
44 <div class="flex justify-center pt-8 px:justify:space-between">
45 <div class="flex justify-center pt-8 px:justify:space-between">
46 <div class="flex justify-center pt-8 px:justify:space-between">
47 <div class="flex justify-center pt-8 px:justify:space-between">
48 <div class="flex justify-center pt-8 px:justify:space-between">
49 <div class="flex justify-center pt-8 px:justify:space-between">
50 <div class="flex justify-center pt-8 px:justify:space-between">
51 <div class="flex justify-center pt-8 px:justify:space-between">
52 <div class="flex justify-center pt-8 px:justify:space-between">
53 <div class="flex justify-center pt-8 px:justify:space-between">
54 <div class="flex justify-center pt-8 px:justify:space-between">
55 <div class="flex justify-center pt-8 px:justify:space-between">
56 <div class="flex justify-center pt-8 px:justify:space-between">
57 <div class="flex justify-center pt-8 px:justify:space-between">
58 <div class="flex justify-center pt-8 px:justify:space-between">
59 <div class="flex justify-center pt-8 px:justify:space-between">
60 <div class="flex justify-center pt-8 px:justify:space-between">
61 <div class="flex justify-center pt-8 px:justify:space-between">
62 <div class="flex justify-center pt-8 px:justify:space-between">
63 <div class="flex justify-center pt-8 px:justify:space-between">
64 <div class="flex justify-center pt-8 px:justify:space-between">
65 <div class="flex justify-center pt-8 px:justify:space-between">
66 <div class="flex justify-center pt-8 px:justify:space-between">
67 <div class="flex justify-center pt-8 px:justify:space-between">
68 <div class="flex justify-center pt-8 px:justify:space-between">
69 <div class="flex justify-center pt-8 px:justify:space-between">
70 <div class="flex justify-center pt-8 px:justify:space-between">
71 <div class="flex justify-center pt-8 px:justify:space-between">
72 <div class="flex justify-center pt-8 px:justify:space-between">
73 <div class="flex justify-center pt-8 px:justify:space-between">
74 <div class="flex justify-center pt-8 px:justify:space-between">
75 <div class="flex justify-center pt-8 px:justify:space-between">
76 <div class="flex justify-center pt-8 px:justify:space-between">
77 <div class="flex justify-center pt-8 px:justify:space-between">
78 <div class="flex justify-center pt-8 px:justify:space-between">
79 <div class="flex justify-center pt-8 px:justify:space-between">
80 <div class="flex justify-center pt-8 px:justify:space-between">
81 <div class="flex justify-center pt-8 px:justify:space-between">
82 <div class="flex justify-center pt-8 px:justify:space-between">
83 <div class="flex justify-center pt-8 px:justify:space-between">
84 <div class="flex justify-center pt-8 px:justify:space-between">
85 <div class="flex justify-center pt-8 px:justify:space-between">
86 <div class="flex justify-center pt-8 px:justify:space-between">
87 <div class="flex justify-center pt-8 px:justify:space-between">
88 <div class="flex justify-center pt-8 px:justify:space-between">
89 <div class="flex justify-center pt-8 px:justify:space-between">
90 <div class="flex justify-center pt-8 px:justify:space-between">
91 <div class="flex justify-center pt-8 px:justify:space-between">
92 <div class="flex justify-center pt-8 px:justify:space-between">
93 <div class="flex justify-center pt-8 px:justify:space-between">
94 <div class="flex justify-center pt-8 px:justify:space-between">
95 <div class="flex justify-center pt-8 px:justify:space-between">
96 <div class="flex justify-center pt-8 px:justify:space-between">
97 <div class="flex justify-center pt-8 px:justify:space-between">
98 <div class="flex justify-center pt-8 px:justify:space-between">
99 <div class="flex justify-center pt-8 px:justify:space-between">
100 <div class="flex justify-center pt-8 px:justify:space-between">
```

Berikut hasilnya.

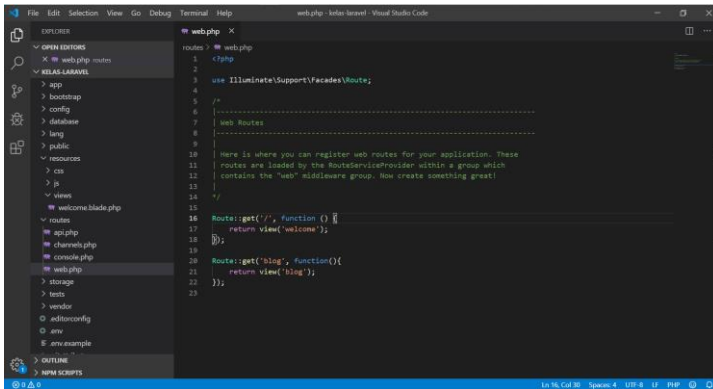


D. Membuat Route & View

Kita akan mencoba membuat **route** dan **view** pada Laravel agar lebih meningkatkan pemahaman kita. Buat **route** baru dengan nama “**blog**” untuk menampilkan **view** “**blog**”.

```
Route::get('blog', function() {  
    return view('blog');  
});
```

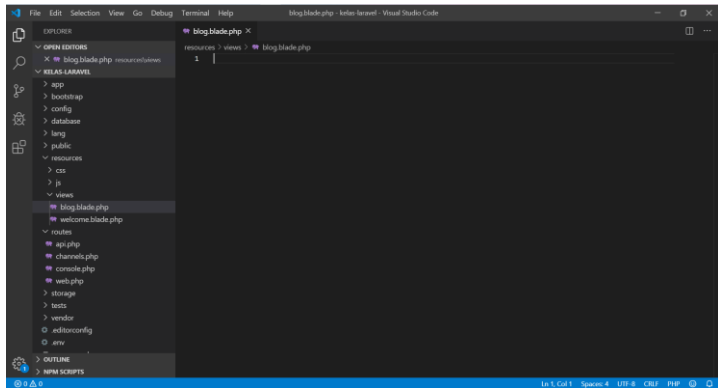
Sehingga, isi lengkap *route* sebagai berikut.



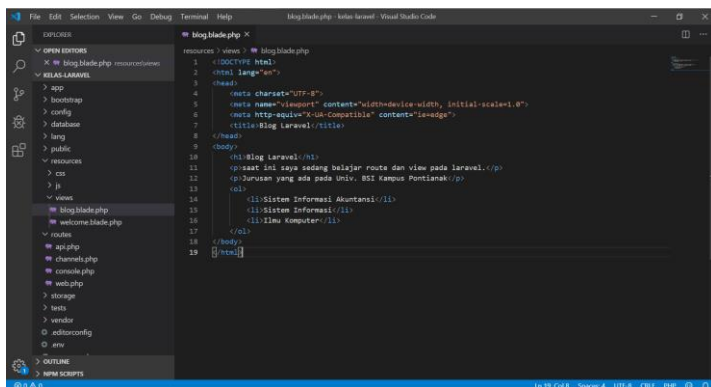
```
1 <?php  
2  
3 use Illuminate\Support\Facades\Route;  
4  
5 /*  
6  * Web Routes  
7  */  
8  
9  
10 Here is where you can register web routes for your application. These  
11 routes are loaded by the RouteServiceProvider within a group which  
12 contains the "web" middleware group. Now create something great!  
13  
14 */  
15  
16 Route::get('/', function () {  
17     return view('welcome');  
18 });  
19  
20 Route::get('blog', function() {  
21     return view('blog');  
22 });  
23
```

Kita sudah membuat *route* yang me-*return* **view** “**blog**”, maka kita harus membuat *file* view pada direktori **resources/views** bernama **blog.blade.php**.



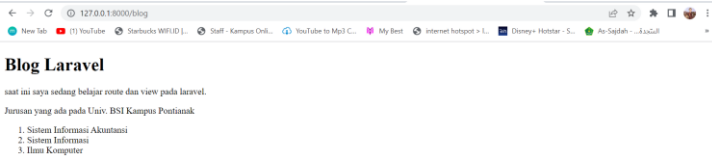


Silakan ketik kode berikut pada **blog.blade.php**.



Untuk melihat hasil dengan mengakses **route** blog pada **http://localhost:8000/blog** akan muncul tampilan dari view **blog.blade.php**. yang kita panggil melalui **route** blog.





E. Latihan

Buatlah **route** baru dengan nama mahasiswa, kemudian tambahkan juga **view** dengan nama **mahasiswa.blade.php** dan buat kode untuk menampilkan tampilan seperti berikut.

Biodata Mahasiswa	
NIM	: 11105601
Nama	: Bang Raje
Jenis Kelamin	: Laki-Laki/Perempuan
Jurusan	: Sistem Informasi Akuntansi, Sistem Informasi, Ilmu Komputer Universitas BSI Pontianak
Alamat	: Subi, Indonesia
Hobi	: • Ngoding • Ngopi • Berenang

Jawab

```
<table border=1>
<tr>
```




```

<td colspan="3" align="center"><b>Biodata Mahasiswa<
/b></td>
</tr>
<tr>
    <td>NIM</td>
    <td>:</td>
    <td>11105601</td>
</tr>
<tr>
    <td>Nama</td>
    <td>:</td>
    <td>Bang Raje</td>
</tr>
<tr>
    <td>Jenis Kelamin</td>
    <td>:</td>
    <td>Laki-Laki/<del>Perempuan</del></td>
</tr>
<tr>
    <td>Jurusan</td>
    <td>:</td>
    <td>Sistem Informasi Akuntansi,<del>Sistem Informasi,
    Ilmu Komputer</del> <br>Universitas BSI Pontianak</
    td>
</tr>
<tr>
    <td>Alamat</td>
    <td>:</td>
    <td>Subi, Indonesia</td>
</tr>

```



```
<tr>
  <td>Hobi</td>
  <td>:</td>
  <td>
    <ul>
      <li>Ngoding</li>
      <li>Ngopi</li>
      <li>Berenang</li>
    </ul>
  </td>
</tr>
</table>
```



BAB 4



MENGENAL VARIABEL PHP DI LARAVEL

Dalam pemrograman, variabel adalah suatu lokasi penyimpanan (di dalam memori komputer) yang berisikan nilai atau informasi yang nilainya tidak diketahui maupun telah diketahui (Wikipedia).

Dalam definisi bebasnya, variabel adalah kode program yang digunakan untuk menampung nilai tertentu. Nilai yang disimpan di dalam variabel selanjutnya dapat dipindahkan ke dalam *database*, atau ditampilkan kembali ke pengguna. Nilai dari variabel dapat diisi dengan informasi yang diinginkan dan dapat diubah nilainya pada saat kode program sedang berjalan.

Sebuah variabel memiliki nama yang digunakan untuk mengakses nilai dari variabel itu. Jika Anda memiliki pengetahuan dasar tentang bahasa pemrograman, tentunya tidak asing dengan istilah variabel.

Sama seperti variabel dalam bahasa pemrograman lainnya, variabel dalam PHP digunakan untuk menampung nilai inputan dari *user*, atau nilai yang kita definisikan sendiri. Namun PHP memiliki beberapa aturan tentang cara penggunaan dan penulisan variabel.



Aturan Penulisan Variabel dalam PHP

Variabel di dalam PHP harus diawali dengan **dollar sign** atau **tanda dollar (\$)**. Setelah tanda \$, sebuah **variabel** PHP harus diikuti dengan karakter pertama berupa huruf atau *underscore* (`_`), kemudian untuk karakter kedua dan seterusnya bisa menggunakan huruf, angka atau *underscore* (`_`). Dengan aturan tersebut, variabel di dalam PHP **tidak bisa** diawali dengan angka. Minimal panjang variabel adalah 1 karakter setelah tanda \$.

Berikut adalah contoh penulisan variabel yang benar dalam PHP:

```
<?php
$i;
$nama;
$Umur;
$_lokasi_memori;
$ANGKA_MAKSIMUM;
?>
```

Dan berikut adalah contoh penulisan variabel yang salah:

```
<?php

$4ever; //variabel tidak boleh diawali dengan angka

$_salah satu; //variabel tidak boleh mengandung spasi

$nama*^; //variabel tidak boleh mengandung karakter khusus: * dan ^
?>
```

PHP membedakan variabel yang ditulis dengan huruf besar dan kecil (bersifat *case sensitif*),



sehingga **\$belajar** tidak sama dengan **\$Belajar** dan **\$BELAJAR**, ketiganya akan dianggap sebagai variabel yang berbeda. Untuk menghindari kesalahan program yang dikarenakan salah merujuk variabel, disarankan menggunakan huruf kecil untuk seluruh nama variabel.

```
<?php
    $rjd="Bang Raje";

echo $rjd; // Notice: Undefined variable: And
i
?>
```

Sama seperti sebagian besar bahasa pemrograman lainnya, untuk memberikan nilai kepada sebuah variabel, PHP menggunakan tanda sama dengan (=). Operator 'sama dengan' ini dikenal dengan istilah Assignment Operators. Perintah pemberian nilai kepada sebuah variabel disebut dengan *assignment*. Jika variabel tersebut belum pernah digunakan, dan langsung diberikan nilai awal, maka disebut juga dengan proses inisialisasi.

Berikut contoh cara memberikan nilai awal (inisialisasi) kepada variabel:

```
<?php
    $nama = "bang raje";
    $umur = 17;

    $pesan = "Saya sedang belajar PHP di duniailk
om.com";
?>
```

variabel tidak perlu deklarasi terlebih dahulu. Anda bebas membuat variabel baru di tengah-tengah kode



program, dan langsung menggunakannya tanpa di deklarasikan terlebih dahulu.

```
<?php
    $rjd="bang raje";
    Echo"$rjd";
?>
```



BAB 5



CONTROLLER PADA LARAVEL

A. Apa itu Controller?

Controller dalam konsep MVC merupakan jembatan penghubung antara **view** dan **model**. Sederhananya controller bisa kita pahami sebagai pengatur View dan Model. Di dalam controller inilah kita dapat membuat penerapan logika atau pengolahan data, kemudian hasil dari controller tersebut ditampilkan pada view.

B. Cara Membuat Controller di Laravel

Ada 2 cara membuat controller pada Laravel, yang pertama kita bisa membuat controller Laravel dengan cara membuat langsung *file* controller baru dalam folder **app/Http/Controllers/**. Kemudian cara yang kedua kita bisa menggunakan perintah php artisan dari Laravel.

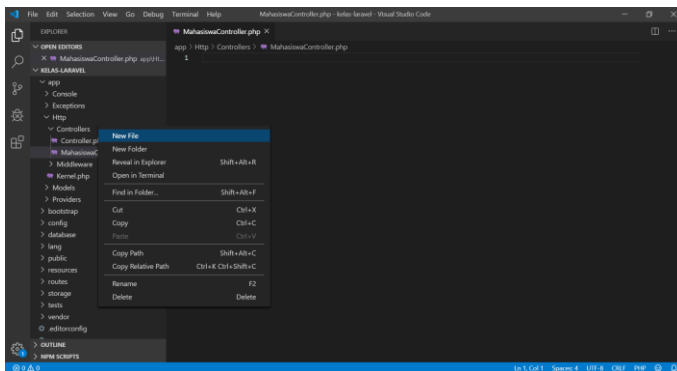
1. Membuat Controller dengan *File* Baru

Kita akan mencoba cara yang pertama, yaitu membuat langsung *file* controller pada direktori **app/Http/Controllers**. Aturan penamaan



controller pada Laravel adalah dengan membuat huruf kapital di awal nama controller.

Misal kita akan membuat controller dengan nama controller-nya adalah **mahasiswa**, maka buat *file* baru dengan nama **MahasiswaController.php** dalam folder **app/Http/Controllers**.



Kemudian buat kode berikut pada **MahasiswaController.php** seperti berikut.

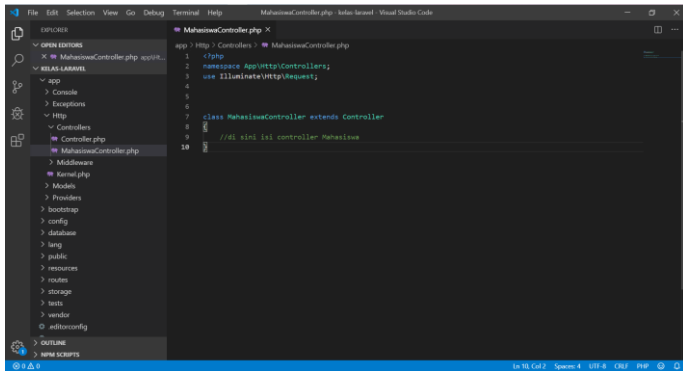
```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;

class MahasiswaController extends Controller
{
    //di sini isi controller Mahasiswa
}
```

Perhatikan kode di atas, kita mendeklarasikan class controller **MahasiswaController** dengan wajib meng-*extends* Controller dari Laravel. Perhatikan juga



penulisan class-nya, penulisan class controller-nya harus sama dengan nama *file* controller yang kita buat.



```
1 <?php
2 namespace App\Http\Controllers;
3 use Illuminate\Http\Request;
4
5
6
7 class MahasiswaController extends Controller
8 {
9     //di sini isi controller Mahasiswa
10 }
```

Sampai di sini kita telah selesai membuat controller pada Laravel dengan cara pertama. Kita lanjut membuat controller dengan cara kedua.

2. Membuat Controller dengan Laravel PHP Artisan

Cara kedua untuk membuat controller pada Laravel adalah kita bisa membuat controller Laravel dengan mudah dengan menggunakan **command php artisan**. **Php artisan** adalah fitur unggulan Laravel yang dibuat untuk memudahkan pengembangan aplikasi yang kita buat. Ada banyak sekali perintah yang bisa digunakan pada php artisan, untuk membuktikan, jalankan perintah php artisan pada **terminal/command prompt** pada direktori *project* Laravel.



```
MINGW64/c:/xampp/htdocs/kelas-laravel

bangr@DESKTOP-HG3HRD9 MINGW64 ~/OneDrive/Desktop
$ cd C:/xampp/htdocs

bangr@DESKTOP-HG3HRD9 MINGW64 /c:/xampp/htdocs
$ ls
applications.html  bitnami.css  img/          webalizer/
belajar-html/     dashboard/  index.php     xampp/
belajar-laravel/  favicon.ico  kelas-laravel/

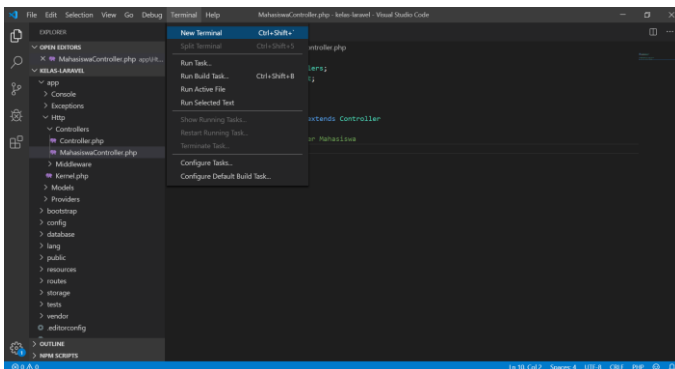
bangr@DESKTOP-HG3HRD9 MINGW64 /c:/xampp/htdocs
$ cd kelas-laravel

bangr@DESKTOP-HG3HRD9 MINGW64 /c:/xampp/htdocs/kelas-laravel
$ php artisan serve

INFO Server running on [http://127.0.0.1:8000].

Press Ctrl+C to stop the server

2022-09-23 09:38:24 ..... ~ 3s
2022-09-23 09:38:24 /favicon.ico ..... ~ 3s
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\xampp\htdocs\kelas-laravel> php artisan
Laravel Framework 9.31.0

Usage:
  command [options] [arguments]

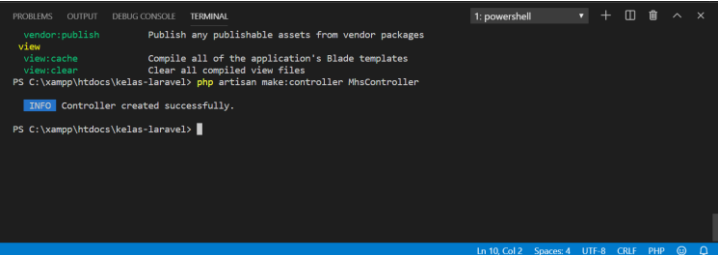
Options:
  -h, --help            Display help for the given command. When no command is given display help for the list command
  -q, --quiet           Do not output any message
  -v, --verbose         Display this application version
                        Force (or disable) ANSI output
  -n, --no-interaction Do not ask any interactive question
  -e[=ENV]             The environment the command should run under
  -v|vv|vvv, --verbose Increase the verbosity of messages: 1 for normal output, 2 for more verbose output and 3 for debug

Available commands:
  about                Display basic information about your application
  clear-compiled       Remove the compiled class file
  completion          Dump the shell completion script
  db                  Start a new database CLI session
  docs                Access the Laravel documentation
  down                Put the application into maintenance / demo mode
  env                Display the current framework environment
  help                Display help for a command
  inspire             Display an inspiring quote
  list                List commands
  migrate              Run the database migrations
  optimize             Cache the framework bootstrap files
```



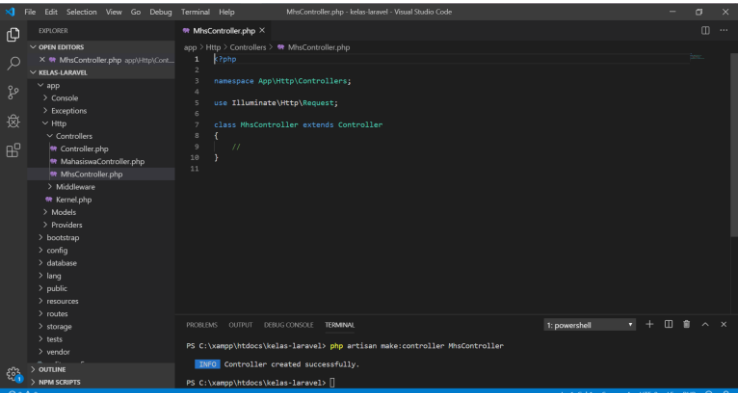
Akan ada penjelasan lebih banyak lagi pada tutorial selanjutnya tentang php artisan. Sekarang kita akan mencoba membuat controller dengan perintah php artisan make:controller. Controller yang akan kita buat bernama **MhsController**. Silakan masuk ke direktori *project* Laravel dan jalankan *command* berikut ini:

php artisan make:controller MhsController



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
1: powershell
vendor:publish Publish any publishable assets from vendor packages
view view:cache Compile all of the application's Blade templates
view:clear Clear all compiled view files
PS C:\xampp\htdocs\kelas-laravel> php artisan make:controller MhsController
INFO Controller created successfully.
PS C:\xampp\htdocs\kelas-laravel>
```

Perintah **make:controller** adalah perintah untuk membuat controller baru, sedangkan **MhsController** adalah nama controller. Maka **controller MhsController.php** akan dibuat secara otomatis.



```
File Edit Selection View Go Debug Terminal Help
MhsController.php - kelas-laravel - Visual Studio Code
EXPLORER
  app > Http > Controllers > MhsController.php
  1 |<?php
  2
  3 namespace App\Http\Controllers;
  4
  5 use Illuminate\Http\Request;
  6
  7 class MhsController extends Controller
  8 {
  9     //
  10 }
  11
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
1: powershell
PS C:\xampp\htdocs\kelas-laravel> php artisan make:controller MhsController
INFO Controller created successfully.
PS C:\xampp\htdocs\kelas-laravel>
```



Kita telah selesai belajar membuat controller Laravel dengan kedua cara di atas. Jika dibandingkan dengan cara pertama tadi, cara kedua sangat mudah sehingga paling direkomendasikan untuk membuat controller Laravel menggunakan php artisan.

C. Cara Menggunakan Controller di Laravel

Selanjutnya kita akan belajar menggunakan Controller Laravel. Pertama kita harus membuat *route* baru yang digunakan untuk mengakses controller. Buka *file web.php* pada folder *routes*, tambahkan baris kode berikut ini paling atas:

```
use App\Http\Controllers\MhsController;
```

Kemudian buat *route* baru untuk mengakses **MhsController**, tambahkan kode berikut ini di paling bawah:

```
Route::get('mhs', [MhsController::class, 'index']);
```

Perhatikan kode di atas, **maksudnya** adalah pada saat url mhs diakses pada web browser, maka **method/function** index pada MhsController akan dijalankan.

Kode lengkap *route* kita pada *file* `web.php` akan seperti berikut:

```
<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\MhsController;

/*
|-----
|
```



```
| Web Routes
|-----
|
| Here is where you can register web routes f
or your application. These
| routes are loaded by the RouteServiceProvider
within a group which
| contains the "web" middleware group. Now cr
eate something great!
|
*/

Route::get('/', function () {
    return view('welcome');
});

Route::get('blog', function() {
    return view('blog');
});

Route::get('Mhs', [MhsController::class, 'ind
ex']);
```

Langkah selanjutnya kita buat **function index** dalam **MhsController** yang sudah kita buat. Buka **file MhsController.php** dan isikan seperti kode di bawah ini:

```
public function index()
{
    return view('mhs');
}
```

Sehingga kode lengkap dari **MhsController.php** akan seperti berikut ini:

```
<?php

namespace App\Http\Controllers;
```



```

use Illuminate\Http\Request;

class MhsController extends Controller
{
    public function index()
    {
        return view('mhs');
    }
}

```

Perhatikan kode di atas, function **index** akan me-return view mhs. Karena view mhs belum ada maka kita buat *file* view **mhs.blade.php** pada **folder resource/views**, berikut isi dari *file* view **mhs.blade.php**:

```

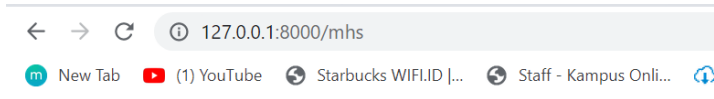
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-
width, initial-scale=1.0">
    <meta http-equiv="X-UA-
Compatible" content="ie=edge">
    <title>kelas Laravel</title>
</head>
<body>
    <h1>Belajar Controller di Laravel</h1>

    <p>Saya sedang belajar membuat controller di
Laravel, mudah dan praktis</p>
    <b>By. Bang Raje</b>
</body>
</html>

```

Simpan dan akses *route* mhs menggunakan alamat **http://localhost:8000/mhs** pada web browser, hasilnya seperti tampilan di bawah ini:





Belajar Controller di Laravel

Saya sedang belajar membuat controller di laravel, mudah dan praktis

By. Bang Raje

D. Latihan

Buatlah kode untuk tampilan berikut pada `mhs.blade.php`

TABEL DATA MAHASISWA				
NIM	NAMA	ALAMAT	TELP	TTL
12106773	Yudhistira	Jl. Perumnas 1 Tangerang	089755767565	Padang, 15 Agustus 1990
1210899	Adi Supriyatna	Jl. Raya Jatiwaringin No.8	085697613939	Jakarta, 17 Oktober 1985
1210893	Putri Rahmawati	Jl. Raya Kramat No. 98	085645612342	Jakarta, 20 Oktober 1986





BAB 6



PASSING DATA CONTROLLER KE VIEW

Passing data adalah proses mengirim/mengoper data. Data yang dikirim bisa berupa Data String dan bisa juga data berupa Data Array. *Passing Data* Laravel adalah bagaimana cara mengirim data dari Controller Laravel untuk ditampilkan di View Laravel.

A. *Passing Data* dari Controller ke View

Sebelumnya kita sudah memiliki MahasiswaController pada bagian controller. Langkah berikutnya yaitu pada membuat *route* baru di **web.php** dengan mengisi kode berikut:

```
use App\Http\Controllers\MahasiswaController;
```

dan kode berikut.

```
Route::get('mahasiswa', [MahasiswaController::  
:class, 'index']);
```

Berikut kode lengkap **route** pada web.php

```
<?php
```

```
use Illuminate\Support\Facades\Route;  
use App\Http\Controllers\MhsController;
```



```

use App\Http\Controllers\MahasiswaController;

/*
|-----
|
| Web Routes
|-----
|
| Here is where you can register web routes f
or your application. These
| routes are loaded by the RouteServiceProvider
within a group which
| contains the "web" middleware group. Now cr
eate something great!
|
*/

Route::get('/', function () {
    return view('welcome');
});

Route::get('blog', function(){
    return view('blog');
});
Route::get('mhs', [MhsController::class, 'ind
ex']);
Route::get('mahasiswa', [MahasiswaController:
:class, 'index']);

```

Kita sudah berhasil membuat controller dan *routing*-nya. Sekarang kita buat function **index** dalam MahasiswaController. Dalam function inilah kita mengirim data ke **view** nantinya. Buka *file MahasiswaController.php* dan buat kode seperti di bawah ini:

```

<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;

```



```

class MahasiswaController extends Controller
{
    public function index()
    {
        $nama = 'Bang Rajee';
        return view('mahasiswa', ['nama' => $nama]);
    }
}

```

Perhatikan kode di atas, kita sudah membuat variabel nama (**\$nama**) yang menyimpan teks **"Bang Rajee"**. Kemudian pada bagian **['nama' => \$nama]** memerintahkan untuk mengirimkan data bernama **\$nama** ke view mahasiswa. Selanjutnya pada view **mahasiswa.blade.php** kita bisa langsung menampilkan data yang ada pada **\$nama** tersebut. Buka *file* view **mahasiswa.blade.php** yang sudah kita buat pada tutorial sebelumnya berada di folder **resource/views**. Maka cara penulisan untuk menampilkan **\$nama** pada *file* view **mahasiswa.blade.php** adalah sebagai berikut:

```

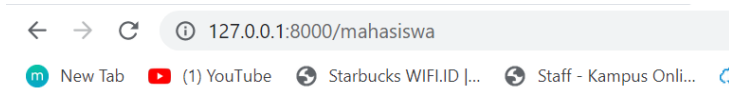
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-
width, initial-scale=1.0">
    <meta http-equiv="X-UA-
Compatible" content="ie=edge">
    <title>Kelas Laravel</title>
</head>
<body>
    <h2>Belajar Passing Data</h2>
    <p>Nama : {{ $nama }}</p>
</body>
</html>

```



Perhatikan kode di atas, kita bisa langsung menampilkan data yang ada dalam \$nama tadi hanya dengan menuliskan kode `{{ $nama }}`. Karena datanya bukan array, kita bisa menampilkan data tersebut tanpa foreach. Inilah kelebihan dari sistem *template blade* dari Laravel (**.blade.php**) yang sangat memudahkan kita. Jadi kita tidak perlu lagi menuliskannya dengan cara membuka *tag php* seperti `<?php echo $nama; ?>`

Sekarang coba buka di web browser dengan url **localhost:8000/mahasiswa**, maka hasilnya seperti berikut ini:



Belajar Passing Data

Nama : Bang Rajee

B. *Passing Data Array*

Untuk *Passing Data Array* Laravel tidak ada perbedaan dengan *Passing Data String* pada cara sebelumnya di atas. Hanya saja jika Data Array yang dikirim lebih dari satu variabel, maka kita tinggal menambahkan simbol koma untuk memisahkan variabel yang dikirim.

Coba Tambahkan function baru dengan nama function **biodata** pada controller **MahasiswaController.php** seperti berikut ini:



```

public function biodata()
{
    $nama = 'Bang rajee';

    $matakuliah = ['Pemrograman Akuntansi 1', 'La
    ravel', 'Bahasa Indonesia'];

    return view('biodata', ['nm' => $nama, 'makul
    ' => $matakuliah]);
}

```

Perhatikan kode di atas, pada bagian **\$matakuliah = ['Pemrograman Akuntansi 1', 'Laravel', 'Bahasa Indonesia']** data array yang kita masukkan dalam **\$matakuliah** bisa langsung kita *passing* ke **view**, dengan cara data-data yang dikirim ke view tinggal kita pisahkan dengan tanda koma seperti kode di atas.

Pada bagian **'makul' => \$matakuliah**, data array yang kita masukkan dalam **\$matakuliah** kemudian kita kirimkan dengan nama **"makul"** ke view **biodata**. Jadi pada view kita akan mengakses dengan nama **"makul"**. Kode lengkap dari *file* BiodataController.php menjadi seperti berikut ini:

```

<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;

class MahasiswaController extends Controller
{
    public function index()
    {
        $nama = 'Bang Rajee';
        return view('mahasiswa', ['nama' => $nama]);
    }
}

```



```

public function biodata()
{
    $nama = 'Bang rajee';

    $matakuliah = ['Pemrograman Akuntansi 1', 'La
    ravel', 'Bahasa Indonesia'];

    return view('biodata', ['nm' => $nama, 'makul
    ' => $matakuliah]);
}
}

```

Karena view biodata belum ada, buat sebuah *file* view baru pada folder **resource/views**, dengan nama **biodata.blade.php**, isi dengan kode berikut:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-
    width, initial-scale=1.0">
    <meta http-equiv="X-UA-
    Compatible" content="ie=edge">
    <title>Kelas Laravel</title>
</head>
<body>
    <h2>Belajar Passing Data Array</h2>
    <p>Nama : {{$nm}}</p>
    <p>Mata Kuliah yang di Ambil :</p>
    <ul>
        @foreach ($makul as $m)
            <li>{{$m}}</li>
        @endforeach
    </ul>

</body>
</html>

```



Perhatikan pada kode di atas, data array makul yang dikirimkan dari controller kita tampilkan dengan menggunakan **foreach()** karena datanya dalam bentuk array. Data **\$makul** kita ubah menjadi **\$m** dalam fungsi **foreach()**, kemudian kita tinggal menampilkan **\$m** seperti pada kode di atas. Penulisan fungsi **foreach** dan perulangan lainnya juga tidak lagi memakai *tag php*, kita bisa menggunakan simbol **@** secara langsung.

Lalu kita butuh *route* baru lagi untuk mengakses function biodata yang berada pada **MahasiswaController.php**. Tambahkan *route* baru pada *file web.php* seperti berikut ini:

```
Route::get('biodata', [MahasiswaController::class, 'biodata']);
```

Sehingga isi *route* lengkap kita menjadi seperti di bawah ini:

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\MhsController;
use App\Http\Controllers\MahasiswaController;

/*
|-----
|-----
| Web Routes
|-----
|-----
|
| Here is where you can register web routes f
or your application. These
```



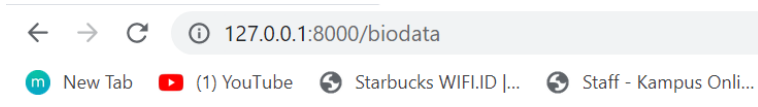
```
| routes are loaded by the RouteServiceProvider
| er within a group which
| contains the "web" middleware group. Now cr
| eate something great!
|
*/

Route::get('/', function () {
    return view('welcome');
});

Route::get('blog', function() {
    return view('blog');
});

Route::get('mhs', [MhsController::class, 'index']);
Route::get('mahasiswa', [MahasiswaController::class, 'index']);
Route::get('biodata', [MahasiswaController::class, 'biodata']);
```

Sekarang lihat hasilnya di **127.0.0.1:8000/biodata**



Belajar Passing Data Array

Nama : Bang rajee

Mata Kuliah yang di Ambil :

- Pemrograman Akuntansi 1
- Laravel
- Bahasa Indonesia



BAB 7



REQUEST DATA DI LARAVEL

Request data adalah proses menerima dan menangkap data. Dalam bahasa pemrograman method **request** data yang biasa kita gunakan adalah method **GET** dan **POST**. Perintah tersebut berfungsi untuk menerima dan menangkap data yang kita kirim dari *form* input dan dari URL.

Dalam Laravel cara yang kita gunakan untuk menangkap data dikenal dengan istilah **Request**. Ada dua proses *request* data atau penerimaan data pada Laravel, yaitu:

1. Menerima Data dari URL Laravel
2. Menerima Data dari *Form* Input Laravel

A. Request Data dari URL

Pertama kita akan belajar menangkap data dari URL. Silakan buka *file* **web.php** yang berada dalam folder *routes*. Buatlah *route* baru seperti di bawah ini:

```
Route::get('mhs/{nama}', [MhsController::class, 'getNama']);
```

Perhatikan kode di atas, kita membuat **route** baru dengan parameter pertama kita buat dengan “**mhs**”,



parameter kedua kita menangkap datanya dengan menuliskan syntax **"{nama}"**. Ketika mengakses **localhost:8000/mhs/bangraje univ bsi pontianak**, maka akan menjalankan function **"getNama"** pada **MhsController**.

Sekarang kita akan membuat function pada **MhsController** sesuai dengan nama *route* yang kita buat yaitu **"getNama"**. Silakan Buka *file MhsController* yang sudah kita buat pada tutorial sebelumnya, kemudian tambahkan function berikut:

```
public function getNama($nama)
{
    return $nama;
}
```

Sehingga isi *route* lengkapnya seperti berikut.

```
<?php

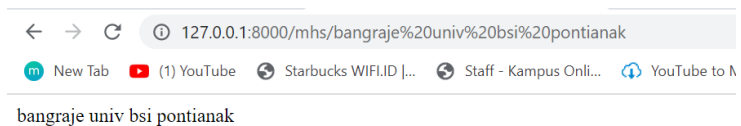
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class MhsController extends Controller
{
    public function index()
    {
        return view('mhs');
    }
    public function getNama($nama)
    {
        return $nama;
    }
}
```



Perhatikan kode di atas, kita telah membuat function “**getNama**” dengan menangkap data yang dikirim dari *route* yang kita buat. Jadi untuk menangkap data dari *route* tadi ({nama}), tinggal kita menuliskan **\$nama** dalam parameter function-nya seperti pada bagian **getNama(\$nama)**, kemudian bisa langsung dapat kita *return* menggunakan variabel yang sama yaitu **\$nama**. Sekarang kita lihat hasilnya, coba jalankan *project* **Laravel** kita kemudian akses **localhost:8000/mhs/bangraje univ bsi pontianak** pada web browser.



B. Request Data dari Inputan

Kita sudah belajar menangkap data dari URL **Laravel**, selanjutnya kita belajar menangkap data dari inputan *form* **Laravel**, kita akan menggunakan method **POST**. Sekarang kita akan membuat dua buah *route* baru, seperti di bawah ini:

```
Route::get('biodata', [MhsController::class, 'biodata']);  
Route::post('biodata/proses', [MhsController::class, 'proses']);
```

Perhatikan kode di atas, pada *route* “**biodata**” kita akan memerintahkan function **biodata** yang ada dalam **MhsController** menggunakan fungsi **get()**. Sedangkan pada *route* “**biodata/proses**” kita memerintahkan



function proses untuk menangkap data yang kita kirim dari *form* menggunakan fungsi **post()**.

Kemudian kita masuk ke controller, silakan buka *file* **MhsController** buat sebuah function dengan nama **biodata**. Pada function **biodata** ini kita memanggil view **biodata** sebagai tampilan untuk *form* inputannya.

Berikut kode dari function **biodata** yang berada dalam **MhsController**:

```
public function biodata()  
{  
    return view(mhs);  
}
```

Pada function **biodata** ini kita memanggil view **mhs**, jadi sekarang kita buka view, Isi dari *file* view **mhs** adalah sebagai berikut:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-  
width, initial-scale=1.0">  
    <meta http-equiv="X-UA-  
Compatible" content="ie=edge">  
    <title>kelas Laravel</title>  
</head>  
<body>  
    <!-- <h1>Belajar Controller di Laravel</h1>  
  
<p>Saya sedang belajar membuat controller di  
Laravel, mudah dan praktis</p>  
    <b>By. Bang Raje</b>  
-->  
<h1>Form Biodata</h1>  
<form action="biodata/proses" method="post">
```



```
{{csrf_field()}}
```

```
nama : <input type="text" name="nama"> <br>  
Kelas : <input type="text" name="kelas"> <br>  
<input type="submit" value="Proses">  
</form>  
</body>  
</html>
```

Perhatikan *form* di atas, pada *form* di atas kita membuat **{{csrf_field}}**. CSRF adalah singkatan dari ***Cross-site request forgery*** yang merupakan sebuah serangan yang dilakukan oleh pengguna yang tidak terautentikasi untuk mengeksekusi perintah. Untuk mengatasi ini, Laravel sudah menyediakan CSRF Token. Nantinya token inilah yang melakukan verifikasi apakah **request** yang diberikan memang berasal dari user yang bersangkutan. Dengan membuat kode **{{csrf_field}}** artinya kita telah membuat sebuah input dengan *type hidden* dan **name_token**, seperti berikut ini:

```
<input          type="hidden"          name="_token"  
value="QQtzCXZYXGeWiPHoK4MJ62rtfSlex9gPqQganb  
v2">
```

Setiap kali kita akan membuat *form*, kita harus memasukkan **{{csrf_field}}** seperti pada kode view **mhs** di atas. Pada *form* yang sudah kita buat di atas, kita mengatur **action**-nya ke “**pendaftaran/proses**” sesuai route yang ada di *file web.php*. Maka, kita akan membuat function lagi dengan nama **proses()** pada *file controller* MhsController. Berikut kode dari function **proses** yang berada dalam MhsController:



```

public function proses(Request $request)
{
    $nama = $request-> nama;
    $kelas = $request-> kelas;

    return 'Nama: ' . $nama. ',<br> Kelas: ' . $kel
as;
}

```

Jadi isi keseluruhan dari
file MhsController.php adalah seperti berikut:

```

<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class MhsController extends Controller
{
    public function index()
    {
        return view('mhs');
    }
    public function getNama($nama)
    {
        return $nama;
    }

    public function biodata()
    {
        return view(mhs);
    }

    public function proses(Request $request)
    {
        $nama = $request-> nama;
        $kelas = $request-> kelas;

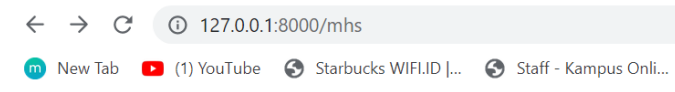
        return 'Nama: ' . $nama. ',<br> Kelas: ' . $kel
as;
    }
}

```



```
}
```

Untuk menangkap data yang dikirim dari *form*, kita bisa menangkap datanya dengan mendeklarasikan “Request” pada parameter function **proses**, dan di lanjutkan dengan menuliskan nama variabel tempat kita menyimpan data-data yang dikirim dari *form* ke dalam sebuah variabel. Pada contoh di atas semua data inputan kita simpan dalam variabel **\$request**. Baru kemudian kita pecah data-data yang kita tangkap dengan menuliskan data apa yang ingin kita ambil (sesuai dengan nama input pada *form* masing-masing). Sekarang coba kita jalankan *project* Laravel dengan mengarah ke **localhost:8000/mhs**



Form Biodata

nama :

Kelas :

Setelah mengisi *form* dengan nama dan alamat, klik tombol **Proses** untuk *submit* data. Kita mengisi dengan Nama **Bang Raje** dan Kelas Laravel 9. Maka hasilnya seperti berikut:





127.0.0.1:8000/biodata/proses



New Tab



(1) YouTube



Starbucks WIFI.ID |...

Nama: Bang Raje,
Kelas: Laravel 9



BAB 8



KONFIGURASI *DATABASE* DI MYSQL

A. Konfigurasi Dasar pada Laravel

Laravel menyediakan sebuah fitur yang sangat memudahkan kita dalam melakukan konfigurasi yaitu *Environmental Configuration*. Setelah melakukan Instalasi, Laravel akan ada sebuah *file* yang bernama **.env** yang terletak di direktori paling luar *project* Laravel kita. Isi *file .env* adalah konfigurasi atau pengaturan-pengaturan *project* Laravel, semua konfigurasi digabung di dalam *file .env* ini.

Tujuan *file .env* agar memudahkan kita dalam mengubah pengaturan yang ingin kita sesuaikan dengan *project* yang ingin kita buat dengan Laravel.

Berikut ini isi default dari *file .env*:

```
APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:ZWp/qOcPraAERFcFfm0MSGqE8278PA
baVQUdNArAyLs=
APP_DEBUG=true
APP_URL=http://localhost

LOG_CHANNEL=stack
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug
```



```
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=kelas_Laravel  
DB_USERNAME=root  
DB_PASSWORD=
```

```
BROADCAST_DRIVER=log  
CACHE_DRIVER=file  
FILESYSTEM_DISK=local  
QUEUE_CONNECTION=sync  
SESSION_DRIVER=file  
SESSION_LIFETIME=120
```

```
MEMCACHED_HOST=127.0.0.1
```

```
REDIS_HOST=127.0.0.1  
REDIS_PASSWORD=null  
REDIS_PORT=6379
```

```
MAIL_MAILER=smtp  
MAIL_HOST=mailhog  
MAIL_PORT=1025  
MAIL_USERNAME=null  
MAIL_PASSWORD=null  
MAIL_ENCRYPTION=null  
MAIL_FROM_ADDRESS="hello@example.com"  
MAIL_FROM_NAME="${APP_NAME}"
```

```
AWS_ACCESS_KEY_ID=  
AWS_SECRET_ACCESS_KEY=  
AWS_DEFAULT_REGION=us-east-1  
AWS_BUCKET=  
AWS_USE_PATH_STYLE_ENDPOINT=false
```

```
PUSHER_APP_ID=  
PUSHER_APP_KEY=  
PUSHER_APP_SECRET=  
PUSHER_HOST=  
PUSHER_PORT=443  
PUSHER_SCHEME=https
```

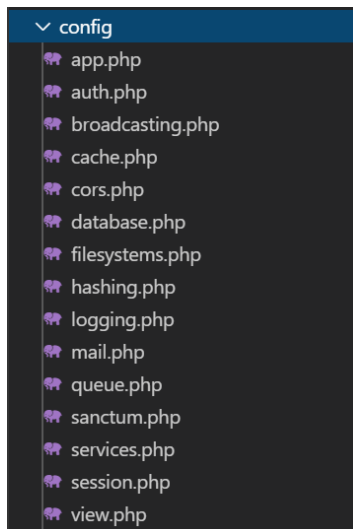


```
PUSHER_APP_CLUSTER=mt1
```

```
VITE_PUSHER_APP_KEY="${PUSHER_APP_KEY}"  
VITE_PUSHER_HOST="${PUSHER_HOST}"  
VITE_PUSHER_PORT="${PUSHER_PORT}"  
VITE_PUSHER_SCHEME="${PUSHER_SCHEME}"  
VITE_PUSHER_APP_CLUSTER="${PUSHER_APP_CLUSTER}"  
}"
```

Sebenarnya semua konfigurasi dari Laravel berada dalam folder **config** dan terpisah-pisah menjadi beberapa *file*. Misalnya untuk konfigurasi *database* pada Laravel ada di *file database.php*, untuk konfigurasi *mailing* ada di *file mail.php*, untuk konfigurasi aplikasi ada dalam folder **app.php**, dan pengaturan-pengaturan lainnya.

Berikut ini isi dari folder **config** Laravel:



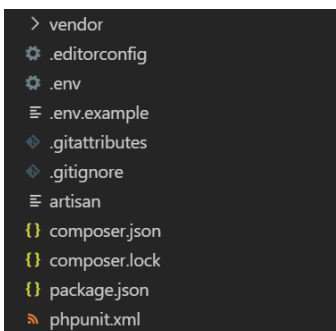
Jadi dengan ada nya *file .env* ini, semua pengaturan berada dalam satu buah *file* ini saja, sehingga



kita tidak perlu berpindah-pindah *file* jika ingin mengubah-ubah pengaturan *project* Laravel.

Misalnya jika kita mengatur *database* dengan nama “**penjualan**” di dalam *file config/database.php*. Kemudian kita juga mengatur *database* dengan nama “**perpustakaan**” di *file .env*, maka *database* perpustakaan yang di *file .env* yang akan digunakan. Karena konfigurasi yang ada dalam *file .env* lebih diutamakan dibandingkan pengaturan yang ada pada folder **config**. Kecuali jika *file .env* kita hapus, maka barulah pengaturan di dalam folder **config** yang akan dijalankan.

Perhatikan *file .env* pada gambar berikut:



Terdapat dua buah *file .env* namun satunya dengan nama **.env.example**. *File example* tersebut sebenarnya adalah *file backup* saja jika suatu saat kita salah mengubah *file .env*. Dengan adanya *file example*, kita bisa mengembalikan *settingan* ke *default* dengan cara melihat isi dari *file .env.example*.

B. **Caching Konfigurasi**

Untuk mempercepat *loading* aplikasi yang kita buat, kita dapat men-*cache* semua konfigurasi yang telah kita *setting*



menjadi dalam satu *file*. Ini akan menggabungkan semua opsi konfigurasi menjadi satu *file* dan dapat dimuat lebih cepat oleh *framework*. Untuk menjalankan perintah *cache* dapat menggunakan perintah php artisan seperti berikut ini:

```
php artisan config:cache
```

Setelah perintah di atas dijalankan, *file .env* tidak akan lagi dimuat karena Laravel sudah memuat hasil *cache* konfigurasi. Namun sebaiknya konfigurasi ini dijalankan hanya ketika aplikasi yang telah kita buat sudah dalam status **production**, bukan masih tahap *development* yang mana kita masih sering mengubah-ubah *file* konfigurasi.

Sedangkan untuk membersihkan *cache*, kamu dapat menggunakan perintah berikut:

```
php artisan config:clear
```

C. *Debug Mode*

Di dalam konfigurasi **.env** pada Laravel, terdapat variable **APP_DEBUG** yang bernilai *default true*. Dalam tahap *development*, tetap biarkan **APP_DEBUG** bernilai **true** agar pesan-pesan eror yang terjadi dalam tahap *development* terlihat oleh kita. Namun jika aplikasi sudah dalam **production**, sebaiknya kita mengatur **APP_DEBUG** menjadi **false** agar informasi sensitif yang ada dalam konfigurasi kita tidak terlihat oleh pengunjung.

D. *Maintenance Mode*

Berbeda dengan *framework* PHP lain, Laravel memiliki kelebihan dengan fitur yang keren, salah satunya

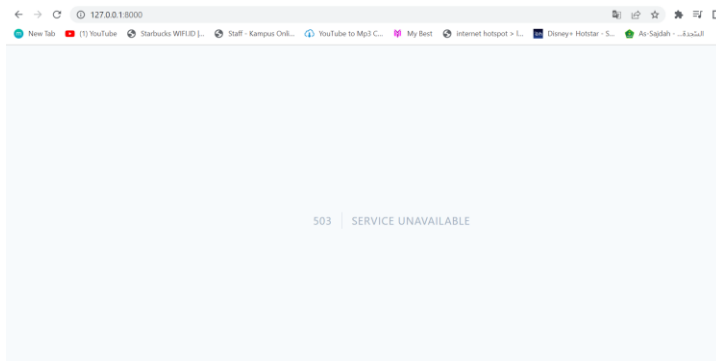


adalah mode *maintenance*. Jika kita mengembangkan aplikasi menggunakan Laravel, kita bisa membuat status aplikasi menjadi *maintenance* atau dalam masa pengembangan. Dengan begitu pengunjung akan tahu bahwa aplikasi sedang dalam pengembangan atau perbaikan.

Untuk mengaktifkan fitur *maintenance*, kita bisa mengaktifkannya dengan salah satu perintah php artisan berikut ini:

```
php artisan down
```

Dengan menjalankan perintah di atas, maka akan muncul pemberitahuan bahwa aplikasi sekarang sedang dalam mode *maintenance*. Akan tampil halaman *simple* dengan pemberitahuan pesan HTTP 503 (*Service Unavailable*) seperti di bawah ini.



Untuk menonaktifkan mode *maintenance*, ketikkan perintah di bawah ini:

```
php artisan up
```

Dengan menjalankan perintah di atas, maka akan muncul pemberitahuan bahwa aplikasi sekarang sudah **live** kembali. Sekarang *project* kita telah aktif kembali karena mode *maintenance* Laravel sudah kita nonaktifkan.

```
PS C:\xampp\htdocs\kelas-laravel> php artisan down
INFO Application is now in maintenance mode.
PS C:\xampp\htdocs\kelas-laravel> php artisan up
INFO Application is now live.
```





BAB 9



SIMPLE TEMPLATE DI LARAVEL

A. Membuat Sistem *Blade Simple Template*

Pada tutorial Belajar Sistem *Blade Template* Laravel ini kita akan membuat halaman web sederhana yang memiliki 3 buah halaman dinamis, yaitu halaman **home**, halaman tentang, dan halaman kontak. Karena kita akan membuat 3 buah halaman **view**, maka kita juga butuh 3 route untuk mengakses halaman-halaman tersebut. Sekarang silakan buat 3 buah *route* pada folder *routes/web.php*.

```
Route::get('blog', [BlogController::class, 'index']);
Route::get('tentang', [BlogController::class, 'tentang']);
Route::get('kontak', [BlogController::class, 'kontak']);
```

Berikut kode lengkapnya.

```
<?php
```

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\MhsController;
use App\Http\Controllers\MahasiswaController;
```



```

use App\Http\Controllers\BlogController;

/*
|-----
|
| Web Routes
|-----
|
| Here is where you can register web routes f
or your application. These
| routes are loaded by the RouteServiceProvider
within a group which
| contains the "web" middleware group. Now cr
eate something great!
|
*/

Route::get('/', function () {
    return view('welcome');
});

Route::get('blogi', function(){
    return view('blog');
});

Route::get('mhs', [MhsController::class, 'ind
ex']);
Route::get('mahasiswa', [MahasiswaController:
:class, 'index']);
Route::get('biodata', [MahasiswaController::c
lass, 'biodata']);
Route::get('mhs/{nama}', [MhsController::class
, 'getNama']);

Route::get('biodata', [MhsController::class,
'biodata']);
Route::post('biodata/proses', [MhsController::
class, 'proses']);

Route::get('blog', [BlogController::class, 'i
ndex']);

```



```
Route::get('tentang', [BlogController::class,
    'tentang']);
Route::get('kontak', [BlogController::class,
    'kontak']);
```

Perhatikan *route* di atas, berikut ini penjelasannya:

Pada saat kita mengakses *route* **blog**, maka yang akan dijalankan adalah function **index** yang ada di dalam controller **BlogController**. Pada saat kita mengakses *route* *tentang*, maka yang akan dijalankan adalah function *tentang* yang ada di dalam controller **BlogController**. Pada saat kita mengakses *route* *kontak*, maka yang akan dijalankan adalah function *kontak* yang ada di dalam controller *BlogController*.

Karena *route*-nya akan menjalankan function pada controller *BlogController*, kita belum punya controller yang bernama *BlogController*. Sekarang kita buat dulu controller **BlogController.php**. Buka terminal, masuk ke direktori *project* *Laravel* kita, dan jalankan perintah berikut:

```
php artisan make:controller BlogController
```

Setelah selesai membuat *BlogController.php*, sekarang kita buat function **index**, function *tentang*, function *kontak* di dalam *BlogController*. Berikut ini function pada *BlogController* lengkap dengan pemanggilan **view**-nya masing-masing:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
```



```

class BlogController extends Controller
{
    //fungsi index
    public function index()
    {
        return view('beranda');
    }

    //fungsi tentang
    public function tentang()
    {
        return view('tentang');
    }

    //fungsi kontak
    public function kontak()
    {
        return view('kontak');
    }
}

```

Perhatikan kode di atas, penjelasannya sebagai berikut:

1. Function **index** memanggil view **beranda**
2. Function **tentang** memanggil view **tentang**
3. Function **kontak** memanggil view **kontak**

Sampai di sini kita akan membuat *template* masternya terlebih dulu. *Template* master yang dimaksud di sini adalah *template* induknya, di *template* master inilah nanti kita menampung semua **view**. Buat sebuah view baru di folder **resource/views** dengan nama **master.blade.php**, isinya sebagai berikut:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-
width, initial-scale=1.0">

```



```

<meta http-equiv="X-UA-
Compatible" content="ie=edge">
<title>Kelas Laravel</title>
</head>
<body align="center">
<header>
<h2>Master Simple Blog</h2>
<nav>
  <a href="/blog">BERANDA | </a>
  <a href="/tentang">TENTANG | </a>
  <a href="/kontak">KONTAK</a>
</nav>
</header>
<hr>

<!-- Bagian Judul -->

<p align="right"> <font color="#A9A9A9"><b>@y
ield('judul')</b></font></p>

<!-- Bagian Isi Konten -->
@yield('konten')

<br><br><br>
<hr>
<footer>
<p>Copy Right @Bang Raje</p>
</footer>

</body>
</html>

```

View **master.blade.php** ini kita jadikan sebagai *template* induk, di mana isi dari view master segala sesuatu yang tidak berubah-ubah seperti menu, *footer* dan nama blognya. Jadi sekarang semua view-view yang di panggil pada function pada BlogController akan kita buat dan kita hubungkan dengan *template* view master ini.



Buat 3 buah view baru, yaitu view **home**, view **tentang** dan view **kontak**.

Isi dari view beranda seperti kode di bawah ini:

```
<!--
- Menghubungkan dengan view template master -
->
@extends('master')

<!-- isi bagian judul halaman -->
<!-- cara penulisan isi section yang pendek -
->
@section('judul', 'Halaman Beranda')

<!-- isi bagian konten -->
<!--
- cara penulisan isi section yang panjang -->
@section('konten')

    <p>Selamat datang!</p>

    <p>Ini Adalah Halaman Beranda - Belajar Siste
m Blade Template Laravel</p>

@endsection
```

Isi dari view tentang seperti kode di bawah ini:

```
<!--
- Menghubungkan dengan view template master -
->
@extends('master')

<!-- isi bagian judul halaman -->
<!-- cara penulisan isi section yang pendek -
->
@section('judul', 'Halaman Tentang')

<!-- isi bagian konten -->
```



```

<!--
- cara penulisan isi section yang panjang -->
@section('konten')

    <p>Ini Adalah Halaman Tentang</p>
    <p>

    Lorem ipsum dolor sit amet, consectetur adipi
    sicing elit, sed do eiusmod

    tempor incididunt ut labore et dolore magna a
    liqua. Ut enim ad minim veniam,

    quis nostrud exercitation ullamco laboris nis
    i ut aliquip ex ea commodo

    consequat. Duis aute irure dolor in reprehend
    erit in voluptate velit esse

    cillum dolore eu fugiat nulla pariatur. Excep
    teur sint occaecat cupidatat non

    proident, sunt in culpa qui officia deserunt
    mollit anim id est laborum.
    </p>

@endsection

```

Isi dari view kontak seperti kode di bawah ini:

```

<!--
- Menghubungkan dengan view template master -
->
@extends('master')

<!-- isi bagian judul halaman -->
<!-- cara penulisan isi section yang pendek -
->
@section('judul', 'Halaman Kontak')

<!-- isi bagian konten -->

```



```

<!--
- cara penulisan isi section yang panjang -->
@section('konten')

    <p>Ini Adalah Halaman Kontak</p>

    <table border="1" cellpadding="3" cellspacing="0" align="center">
        <tr>
            <td>Email</td>
            <td>:</td>
            <td>info@gmail.com</td>
        </tr>
        <tr>
            <td>Facebook</td>
            <td>:</td>
            <td>facebook/bangraje</td>
        </tr>
        <tr>
            <td>Twitter</td>
            <td>:</td>
            <td>twitter/bangraje</td>
        </tr>
        <tr>
            <td>Hp</td>
            <td>:</td>
            <td>0853-0835-3555</td>
        </tr>
    </table>

@endsection

```

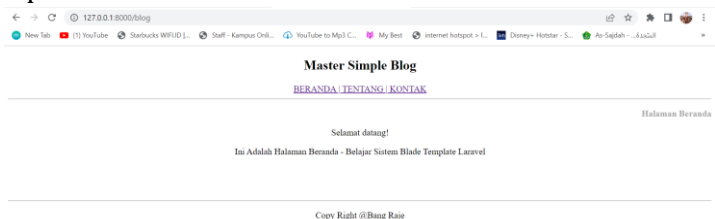
Perhatikan semua kode view di atas, kita telah membuat empat buah *file* view yaitu:

1. View **master.blade.php**: Terdapat fungsi dengan nama **yield()**. Fungsi **@yield()** berguna untuk bagian-bagian tertentu pada *template website* yang isinya dinamis. Isi dari view lain akan kita tampung di dalam parameter dengan fungsi **@yield()**.



2. View **beranda.blade.php**: Terdapat fungsi dengan nama **@extends()** dan **@section()**. Fungsi **@extends()** berfungsi untuk menghubungkan view beranda dengan view master. Kemudian semua isi yang ada di antara fungsi **@section** pada view beranda akan ditampilkan pada bagian fungsi **@yield** pada view master.
3. View **tentang.blade.php**: Terdapat fungsi dengan nama **@extends()** dan **@section()**. Fungsi **@extends()** berfungsi untuk menghubungkan view tentang dengan view master. Kemudian semua isi yang ada di antara fungsi **@section** pada view tentang akan di tampilkan pada bagian fungsi **@yield** pada view master.
4. View **kontak.blade.php**: Terdapat fungsi dengan nama **@extends()** dan **@section()**. Fungsi **@extends()** berfungsi untuk menghubungkan view kontak dengan view master. Kemudian semua isi yang ada di antara fungsi **@section** pada view kontak akan di tampilkan pada bagian fungsi **@yield** pada view master.

Sekarang kita coba akses *project* kita dengan mengakses **localhost:8000/blog**, akan muncul halaman seperti berikut:



B. Latihan

Silakan buat master **simple blog** dengan semenarik mungkin dan isi dengan konten yang bermanfaat. Selamat mencoba!



BAB 10



FUNGSI MATEMATIKA DI LARAVEL

Bahasa pemrograman PHP datang dengan membawa banyak fungsi-fungsi yang berkaitan dengan matematika, seperti fungsi perpangkatan, akar kuadrat, sebagainya. Pada pertemuan kali ini kita akan mempelajari beberapa hal penting tentang fungsi-fungsi matematika pada Laravel.

Pertama kita akan memuat fungsi matematika sederhana seperti penjumlahan, pengurangan, perkalian dan pembagian di Laravel. Seperti biasanya sebelum memulai silakan aktifkan php artisan **serve** pada *project* Laravel kalian masing-masing, atau bisa juga membuat *project* baru. Di sini masih menggunakan *project* lama yaitu kelas Laravel yang pada bab sebelumnya pernah di buat.

Jika *project* Laravel sudah bisa diaktifkan, maka silakan buat kode berikut pada halaman **web.php** yang terletak pada folder **routes**.

```
use App\Http\Controllers\LatihanController;
```

Kemudian setelah itu tambah kode berikut ini untuk penggunaan **view** dan **controller**.

```
Route::get('matematika',[LatihanController::clas  
s, 'index']);
```



Jika sudah, langkah berikutnya adalah membuat controller **LatihanController.php** pada bagian controller Laravel, silakan buat controller tersebut menggunakan **make:controller** yang sudah di bahas pada bab sebelumnya. Kemudian tambah kode seperti berikut ini pada *file LatihanController.php*.

```
public function index()
{
    $nilai1 = 12;
    $nilai2 = 10;
    $hasil_penjumlahan = $nilai1 + $nilai2;
    $hasil_pengurangan = $nilai2 - $nilai2;
    $hasil_perkalian = $nilai2 * $nilai2;
    $hasil_bagi = $nilai2 / $nilai2;

    return view('matematika', ['nml' => $nilai1, 'nm
2' => $nilai2, 'hasil_j' => $hasil_penjumlahan,
'hasil_k' => $hasil_pengurangan, 'hasil_x' => $h
asil_perkalian, 'hasil_b' => $hasil_bagi]);
}
```

Berikut kode lengkapnya.

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class LatihanController extends Controller
{
    //fungsi index
    public function index()
    {
        $nilai1 = 12;
        $nilai2 = 10;
        $hasil_penjumlahan = $nilai1 + $nilai2;
        $hasil_pengurangan = $nilai2 - $nilai2;
```



```

$hasil_perkalian = $nilai2 * $nilai2;
$hasil_bagi = $nilai2 / $nilai2;

return view('matematika', ['nm1' => $nilai1, 'nm
2' => $nilai2, 'hasil_j' => $hasil_penjumlahan,
'hasil_k' => $hasil_pengurangan, 'hasil_x' => $h
asil_perkalian, 'hasil_b' => $hasil_bagi]);
}
}

```

Pada *file* ini kita membuat sebuah fungsi dengan nama **index** yang di dalamnya menampung nilai variable **\$nilai1**, **\$nilai2**, **\$hasil_penjumlahan**, **\$hasil_pengurangan**, **\$hasil_perkalian**, **\$hasil_bagi**. Dan akan di tampilkan di *file* **matematika.blade.php** yang berada di view. Sedangkan **nm1**, **nm2**, **nm3**, **hasil_p**, **hasil_k**, **hasil_x** dan **hasil_b** merupakan deklarasi dari variabel dan akan digunakan pada halaman **matematika.blade.php**.

Agar lebih paham dan jelasnya silakan buat *file* **matematika.blade.php** pada folder **view** di Laravel. Dan masukkan kode berikut ini.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-
width, initial-scale=1.0">
  <meta http-equiv="X-UA-
Compatible" content="ie=edge">
  <title>Kelas Laravel</title>
</head>
<body>
  <h2>Matematika di Laravel</h2>

  <b>Hasil Penjumlahan {{$nm1}} + {{$nm2}} = {{$ha
sil_j}}</b> <br>

```



```
<b>Hasil Penjumlahan {{$nm1}} - {{$nm2}} = {{$hasil_k}}</b> <br>
```

```
<b>Hasil Penjumlahan {{$nm1}} x {{$nm2}} = {{$hasil_x}}</b> <br>
```

```
<b>Hasil Penjumlahan {{$nm1}} : {{$nm2}} = {{$hasil_b}}</b> <br>
```

```
</body>
```

```
</html>
```

Pemanggilan variabel tersebut di tulis di seperti ini
{{\$nm1}} dst, yang di ambil dari halaman
LatihanController.php. berikut hasilnya
<http://127.0.0.1:8000/matematika>



127.0.0.1:8000/matematika



New Tab



(1) YouTube



Starbucks WIFI.ID |...



Staff - Kampus Onli...

Matematika di Laravel

Hasil Penjumlahan $12 + 10 = 22$

Hasil Penjumlahan $12 - 10 = 0$

Hasil Penjumlahan $12 \times 10 = 100$

Hasil Penjumlahan $12 : 10 = 1$

A. Bilangan Berpangkat: Pow()

Fungsi **pow()** bertugas untuk mengangkat suatu bilangan pada PHP. Fungsi ini membutuhkan 2 parameter:

1. Bilangan yang akan dipangkat.
2. Nilai eksponen.

Berikut contoh pembuatannya di Laravel.

Pada *file* **web.php** tambahkan kode berikut.



```
Route::get('pangkat',[LatihanController::class,
'pangkat']);
```

Kemudian tambahkan kode berikut pada **LatihanController.php** sebagai berikut.

```
public function pangkat()
{
    $a = 5; //5 pangkat 2
    $b = 12; //12 pangkat 3
    $c = 10; //10 pangkat 4

    $aa = pow(5,2);
    $bb = pow(12,3);
    $cc = pow(10,4);

    $x = sqrt(25);
    $y = sqrt(4);

    return view('pangkat', ['nilai1'=> $a,'nilai2' =
    > $b, 'nilai3' => $c,

    'pangkat1' => $aa, 'pangkat3' => $bb, 'pangkat4'
    => $cc,'pangkat']);

}
```

Dan jika di lihat secara keseluruhan sebagai berikut kode programnya.

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class LatihanController extends Controller
{
    //fungsi index
```



```

public function index()
{
    $nilai1 = 12;
    $nilai2 = 10;
    $hasil_penjumlahan = $nilai1 + $nilai2;
    $hasil_pengurangan = $nilai2 - $nilai2;
    $hasil_perkalian = $nilai2 * $nilai2;
    $hasil_bagi = $nilai2 / $nilai2;

    return view('matematika', ['nm1' => $nilai1, 'nm
2' => $nilai2, 'hasil_j' => $hasil_penjumlahan,
'hasil_k' => $hasil_pengurangan, 'hasil_x' => $h
asil_perkalian, 'hasil_b' => $hasil_bagi]);
}

public function pangkat()
{
    $a = 5; //5 pangkat 2
    $b = 12; //12 pangkat 3
    $c = 10; //10 pangkat 4

    $aa = pow(5,2);
    $bb = pow(12,3);
    $cc = pow(10,4);

    $x = sqrt(25);
    $y = sqrt(4);

    return view('pangkat', ['nilai1'=> $a,'nilai2' =
> $b, 'nilai3' => $c,

'pangkat1' => $aa, 'pangkat3' => $bb, 'pangkat4'
=> $cc,'pangkat']);
}
}

```

Jika sudah maka kita akan membuat *file* baru lagi pada view dengan nama **pangkat.blade.php** dan masukkan kode berikut ini.




```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-
width, initial-scale=1.0">
  <meta http-equiv="X-UA-
Compatible" content="ie=edge">
  <title>Kelas Laravel</title>
</head>
<body>
  <h2>Matematika (Pangkat) di Laravel</h2>

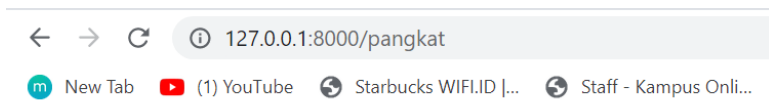
  <b> {{ $nilai1 }} pangkat 2 adalah {{ $pangkat1 }} </b>
  <br>

  <b> {{ $nilai2 }} pangkat 3 adalah {{ $pangkat3 }} </b>
  <br>

  <b> {{ $nilai3 }} pangkat 4 adalah {{ $pangkat4 }} </b> <
  br> <br>
<br>
  <b>By. Bang Raje </b>
</body>
</html>

```

Berikut hasilnya <http://127.0.0.1:8000/pangkat>



Matematika (Pangkat) di Laravel

5 pangkat 2 adalah 25
12 pangkat 3 adalah 1728
10 pangkat 4 adalah 10000

By. Bang Raje



B. Akar Bilangan: `sqrt()`

Fungsi **`sqrt()`** bertugas untuk menghitung akar kuadrat dari suatu bilangan. Dan ditinjau dari segi parameter, ia hanya membutuhkan satu parameter.

Untuk membuat akar bilangan di Laravel bisa kita gunakan *file* **`pangkat.blade.php`** saja agar tidak banyak *file* kita gunakan. Dan kita cukup menambahkan kode berikut di **`web.php`** sebagai berikut.

```
Route::get('pangkat',[LatihanController::clas  
s, 'pangkat']);
```

Kemudian pada *file* **`LatihanContoroller.php`** tambahkan kode berikut ini.

```
public function pangkat()  
{  
    $a = 5; //5 pangkat 2  
    $b = 12; //12 pangkat 3  
    $c = 10; //10 pangkat 4  
  
    $aa = pow(5,2);  
    $bb = pow(12,3);  
    $cc = pow(10,4);  
  
    $x = sqrt(25); //akar dari 25  
    $y = sqrt(4); //akar dari 4  
  
    return view('pangkat', ['nilai1'=> $a,'nilai2'  
    ' => $b, 'nilai3' => $c,  
  
    'pangkat1' => $aa, 'pangkat3' => $bb, 'pangka  
    t4' => $cc,'pangkat',  
    'x'=> $x, 'y' => $y]);  
}
```



Berikut adalah kode lengkapnya pada *file LatihanController.php* seperti berikut ini.

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class LatihanController extends Controller
{
    //fungsi index
    public function index()
    {
        $nilai1 = 12;
        $nilai2 = 10;
        $hasil_penjumlahan = $nilai1 + $nilai2;
        $hasil_pengurangan = $nilai2 - $nilai2;
        $hasil_perkalian = $nilai2 * $nilai2;
        $hasil_bagi = $nilai2 / $nilai2;

        return view('matematika', ['nm1' => $nilai1,
        'nm2' => $nilai2, 'hasil_j' => $hasil_penjumlahan, 'hasil_k' => $hasil_pengurangan, 'hasil_x' => $hasil_perkalian, 'hasil_b' => $hasil_bagi]);
    }

    public function pangkat()
    {
        $a = 5; //5 pangkat 2
        $b = 12; //12 pangkat 3
        $c = 10; //10 pangkat 4

        $aa = pow(5,2);
        $bb = pow(12,3);
        $cc = pow(10,4);

        $x = sqrt(25); //akar dari 25
        $y = sqrt(4); //akar dari 4
    }
}
```



```

return view('pangkat', ['nilai1'=> $a, 'nilai2'
=> $b, 'nilai3' => $c,

'pangkat1' => $aa, 'pangkat3' => $bb, 'pangka
t4' => $cc, 'pangkat',
'x'=> $x, 'y' => $y]);

}
}

```

Langkah berikutnya adalah menambahkan kode pada halaman **pangkat.blade.php** seperti berikut ini.

```

<h2>Matematika (Akar) di Laravel</h2>
<b>Akar 25 adalah {{$x}} </b> <br>
<b>Akar 4 adalah {{$y}} </b>

```

Berikut kode lengkapnya pada *file* **pangkat.blade.php**

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-
width, initial-scale=1.0">
    <meta http-equiv="X-UA-
Compatible" content="ie=edge">
    <title>Kelas Laravel</title>
</head>
<body>
    <h2>Matematika (Pangkat) di Laravel</h2>

    <b> {{$nilai1}} pangkat 2 adalah {{$pangkat1}} <
/b> <br>

    <b> {{$nilai2}} pangkat 3 adalah {{$pangkat3}} <
/b> <br>

    <b> {{$nilai3}} pangkat 4 adalah {{$pangkat4}} </
b> <br> <br>

```



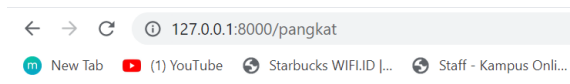
```

<h2>Matematika (Akar) di Laravel</h2>
<b>Akar 25 adalah {{ $x }} </b> <br>
<b>Akar 4 adalah {{ $y }} </b>

<br>
<b>By. Bang Raje </b>
</body>
</html>

```

Berikut hasilnya <http://127.0.0.1:8000/pangkat>



Matematika (Pangkat) di Laravel

5 pangkat 2 adalah 25
 12 pangkat 3 adalah 1728
 10 pangkat 4 adalah 10000

Matematika (Akar) di Laravel

Akar 25 adalah 5
 Akar 4 adalah 2
 By. Bang Raje

C. Latihan

1. Sebuah toko kue selama 8 hari dapat membuat 240 kotak kue. Banyak kue yang dapat dibuat oleh toko tersebut selama 12 hari adalah
2. Pak Arif membeli motor dengan harga Rp15.000.000,00 dan dijual lagi dengan harga Rp16.500.000,00. Berapa persentase keuntungan yang diperoleh?

Kunci jawaban ada di *link* berikut ini.

<https://www.ruangguru.com/blog/latihan-soal-ujian-nasional-2019-matematika-smp>





BAB 11



FUNGSI *FORM* PADA LARAVEL

A. Pengenalan *Form*

Form digunakan untuk mengumpulkan data dari pengunjung web, mulai dari *login form*, pendaftaran dan mengirimkan data antar halaman web. Penggunaan *form* dengan html tidak akan terlalu berguna, *form* biasanya hanya berupa *interface* yang disediakan untuk mengumpulkan data dari user, dan akan diproses dengan bahasa pemrograman web lainnya.

Di dalam tag *form* terdapat atribut yaitu *method*, berfungsi untuk menjelaskan bagaimana data isian *form* akan dikirim oleh web browser. Nilai dari atribut *method* ini bisa berupa “**get**” atau “**post**”. Perbedaan *method get* dan *method post*, jika kita mengisi atribut *method* dengan **get** (*default* seandainya atribut *method* tidak ditulis) maka isian *form* akan terlihat pada url browser-*method get* ini biasanya digunakan untuk query pencarian.

Method post biasanya digunakan untuk data yang lebih sensitif seperti yang berisi *password*, atau registrasi user-data hasil *form* tidak akan terlihat pada browser.



B. Komponen *Form*

Ada beberapa jenis komponen *form* yaitu sebagai berikut.

1. Form : `<FORM ACTION=action METHOD=method>`
`</FORM>`
2. Text Box : `<INPUT TYPE=TEXT`
`NAME="nama_variabel" VALUE="value">`
3. Text Area : `<textarea rows=" " cols=" "`
`name="nama_variabel"> </textarea>`
4. Radio Button : `<input type="radio"`
`name="nama_variabel" value=" " >Isi_Radio`
5. Combo Box : `<select name="nama_variabel" value=" " >`
6. `<option>Combo1</option>`
7. `<option>Combo2</option></select>`
8. Check Box : `<input type="checkbox"`
`name="kendaraan" value="Motor">Motor<br>`
9. `<input type="checkbox" name="kendaraan"`
`value="Mobil"`
10. `checked>Mobil<br>`
11. Submit : `<input type="submit" name="submit"`
`value="submit">`
12. Reset : `<input type="reset" name="reset"`
`value="reset">`

C. Pengolahan Data Form

Form di HTML dikenal dengan adanya *tag* `<FORM>` dan ditutup dengan tag `</FORM>`. Di dalam *tag* pembuka `<FORM>` diikuti dengan atribut **action** dan **method**. **Action** menjelaskan ke halaman yang digunakan untuk memproses input, sementara *method* digunakan untuk mengatur cara mem-*parsing* konten Web menerima input



dari user atau pengunjung menggunakan metode **GET** dan **POST**.

GET akan mengirimkan data Bersama dengan URL, sedangkan **POST** akan mengirimkannya secara terpisah. User mengirimkan data input dengan mengisi teks atau pilihan pada atribut *form* html.

D. Membuat Inputan di Laravel

Langkah awal yang kita lakukan adalah membuat controller dengan nama **BelajarController.php**, gunakan perintah php artisan **make:controller BelajarController** menggunakan terminal. Jika sudah maka masukkan kode berikut ini pada *file BelajarController.php*.

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class BelajarController extends Controller
{
    //fungsi untuk formulir
    public function index()
    {
        return view('formulir');
    }

    public function hasil (Request $request)
    {
        $nim = $request->input('nim');
        $nama = $request->input('nm');
        $salamat = $request->input('almt');
        $ttl = $request->input('tgllhr');
        $gender = $request->input('jk');
        $kerja = $request->input('kerja');
        $email = $request->input('email');
        $hobi1 = $request->input('hobi1');
        $hobi2 = $request->input('hobi2');
        $hobi3 = $request->input('hobi3');
```



```

        return
        "<h2>Data Pendaftaran Kelas Laravel</h2> <br> N
        IM : ".$nim.
        "<br> Nama : ".$nama."<br> Alamat : ".$alamat.

        "Tanggal lahir : ".$ttl."<br> Gender : ".$gende
        r.
        "<br> Pekerjaan : ".$kerja."<br> E-
        mail".$email.
        "<br> Hobi : ".$hobi1.", ".$hobi2.", ".$hobi3.
        "<br><a href='/formulir'>kembali</a>";
    }
}

```

Pada fungsi index kita akan *me-request* halaman formulir, maka kita akan membuat *file* dengan nama **formulir.blade.php** pada bagian view. Kemudian buatlah kode berikut ini pada *file* tersebut.

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">

    <meta name="viewport" content="width=device
    -width, initial-scale=1.0">
    <meta http-equiv="X-UA-
    Compatible" content="ie=edge">
    <title>Kelas Laravel</title>
</head>
<body>

    <form action="/formulir/hasil" method="get"
    >
        <table align="center">
            <tr>

                <td align="right" colspan=3> <h3>Formul
                ir Pendaftaran Kelas Laravel</h3> </td>

```



```

</tr>
<tr>
    <td>NIM</td>
    <td>:</td>
    <td><input type="text" name="nim"></td>
</tr>
<tr>
    <td>NAMA</td>
    <td>:</td>
    <td><input type="text" name="nm"></td>
</tr>
<tr>
    <td>Alamat</td>
    <td>:</td>
    <td><textarea name="almt" id="" cols="30"
rows="10"></textarea></td>
</tr>
<tr>
    <td>Tanggal lahir</td>
    <td>:</td>
    <td><input type="date" name="tgllhr"></td>
</tr>
<tr>
    <td>Gender</td>
    <td>:</td>
    <td>
        <input type="radio" name="jk" value="Laki-
Laki">Laki - Laki<br>
        <input type="radio" name="jk" value="Perem
puan">Perempuan
    </td>
</tr>
<tr>
    <td>Pekerjaan</td>
    <td>:</td>
    <td>
        <select name="kerja">

```



```

        <option value="">--Pilih Pekerjaan--
    </option>
    <option value="Karyawan">Karyawan</option>

    <option value="Mahasiswa">Mahasiswa</option>

    <option value="Wirausaha">Wirausaha</option>

    <option value="Belum Bekerja">Belum Bekerja<
    /option>
</select>
</td>
</tr>
<tr>
    <td>Email</td>
    <td>:</td>

    <td><input type="email" name="email" place
    holder="Email Anda"></td>
</tr>

<tr valign="top">
    <td>Hobi</td>
    <td>:</td>
    <td>

        <input type="checkbox" name="hobi1" value=
        "Olah Raga">Olah Raga<br>

        <input type="checkbox" name="hobi2" value=
        "Musik">Musik<br>

        <input type="checkbox" name="hobi3" value=
        "Renang">Renang
    </td>
</tr>
<tr>

    <td align="center" colspan=3><input type
    ="submit" value="SIMPAN">
    <input type="reset" value="BATAL"></td>
</tr>

```



```

        </table>
    </form>
</body>
</html>

```

Sekarang sebelum melihat hasilnya maka tambahkan kode berikut pada **web.php** (*route*) seperti berikut.

Bagian atas

```
use App\Http\Controllers\BelajarController;
```

Bagian bawah

```
Route::get('formulir', [BelajarController::class, 'index']);
Route::get('formulir/hasil', [BelajarController::class, 'hasil']);
```

Berikut kode keseluruhannya.

```

<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\MhsController;
use App\Http\Controllers\MahasiswaController;
use App\Http\Controllers\BlogController;
use App\Http\Controllers\LatihanController;
use App\Http\Controllers\BelajarController;

/*
|-----
|-----
|  Web Routes
|-----
|-----
|

```



```
| Here is where you can register web routes f
or your application. These
| routes are loaded by the RouteServiceProvider
er within a group which
| contains the "web" middleware group. Now cr
eate something great!
|
*/
```

```
Route::get('/', function () {
    return view('welcome');
});
```

```
Route::get('blogi', function(){
    return view('blog');
});
```

```
Route::get('mhs', [MhsController::class, 'ind
ex']);
Route::get('mahasiswa', [MahasiswaController:
:class, 'index']);
//Route::get('biodata', [MahasiswaController:
:class, 'biodata']);
Route::get('mhs/{nama}', [MhsController::class
, 'getNama']);
```

```
Route::get('biodata', [MhsController::class,
'biodata']);
Route::post('biodata/proses', [MhsController::
class, 'proses']);
```

```
Route::get('blog', [BlogController::class, 'i
ndex']);
Route::get('tentang', [BlogController::class,
'tentang']);
Route::get('kontak', [BlogController::class,
'kontak']);
Route::get('matematika', [LatihanController::c
lass, 'index']);
Route::get('pangkat', [LatihanController::clas
s, 'pangkat']);
```



```
Route::get('formulir', [BelajarController::class, 'index']);
Route::get('formulir/hasil', [BelajarController::class, 'hasil']);
```

Sekarang silakan buka browsernya dan lihat hasilnya <http://127.0.0.1:8000/formulir>

Formulir Pendaftaran Kelas Laravel

NIM	:	11210989
NAMA	:	bang raje
Alamat	:	Subi, Natuna-Indonesia
Tanggal lahir	:	17/08/2000 
Gender	:	☒ Laki - Laki ☐ Perempuan
Pekerjaan	:	Karyawan
Email	:	bangraje@gmail.com
Hobi	:	☒ Olah Raga ☒ Musik ☒ Renang
SIMPAN BATAL		



Data Pendaftaran Kelas Laravel

NIM : 11210989

Nama : bang raje

Alamat : Subi, Natuna-Indonesia Tanggal lahir : 2000-08-17

Gender : Laki-Laki

Pekerjaan : Karyawan

E-mail bangraje@gmail.com

Hobi : Olah Raga, Musik, Renang

[kembali](#)

E. Latihan

Silakan buat *form* inputan pendaftaran perkuliahan yang isinya ada nama, tempat lahir, tanggal lahir, jenis kelamin (L/P), No. Hp, Alamat, Pilih Jurusan (SI, SIA, ILKOM), Biaya Sumbangan Pendidikan, Biaya Semester (1,2,3,4,5,6), Pilih Metode Bayar (Bank BCA, Bank BNI, Indomaret), Tanggal Bayar.



BAB 12



STUDI KASUS SEDERHANA

A. Pembelian Tiket

Untuk studi kasus sederhana yang akan di buat adalah membuat *website* pembelian tiket konser. Di sini masih menggunakan *project* yang lama yaitu kelas-Laravel jadi tidak perlu membuat *project* baru lagi. Tahapan awal pembuatan web pembelian tiket ini yaitu membuat controller terlebih dahulu yaitu dengan nama **BookingController.php** dengan cara ketikan pada terminal yaitu php artisan **make:controller BookingController**, maka program Laravel akan otomatis men-*generate file* BookingController di bagian controller. Jika sudah maka masukkan kode berikut ini pada **BookingController.php**

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class BookingController extends Controller
{
    //fungsi tampilan booking
    public function booking()
    {
        return view('booking');
    }
}
```



```

//fungsi hasil inputan booking
public function hasil(Request $request)
{
    $nama = $request->input('nama');
    $studio = $request->input('studio');
    $kelas = $request->input('kelas');
    $jmltiket = $request->input('jmltiket');
    //Mencari Nama Bintang Tamu
    if ($studio=="Studio 1")
    {
        $bt="Opick";
    }
    elseif ($studio=="Studio 2")
    {
        $bt="Raihan";
    }
    //Mencari Harga
    if($kelas=='VIP'){
        $harga=500000;
    }
    elseif($kelas=='Festival')
    {
        $harga=250000;
    }
    $total=$harga*$jmltiket;

    return view('hasil', compact('nama','studio',
    'bt','kelas','harga','jmltiket','total'));
}
}

```

Setelah selesai membuat controller, Langkah berikutnya adalah membuat view yaitu **booking.blade.php** dan **hasil.blade.php**

Booking.blade.php

```

<!DOCTYPE html>
<html>

```



```

<head>
<title>Form Input Laravel</title>
</head>
<body>
<form action="hasil" method="post">
  {{ csrf_field() }}
  <table bgcolor="#5F9EA0" border="0" align="center" width="60%">
    <tr>

<td colspan="3" align="center"><h2>PEMBELIAN
TIKET KONSER AMAL</h2><hr></td>
    </tr>
    <tr>
    <td>Nama</td>
    <td>:</td>

<td><input type="text" name="nama" placeholder="Masukkan Nama anda" size="40"></td>
    </tr>
    <tr valign="top">
    <td>Studio</td>
    <td>:</td>
    <td>
      <select name="studio">
        <option value="">--Pilih Studio--</option>
        <option value="Studio 1">Studio 1</option>
        <option value="Studio 2">Studio 2</option>
      </select>
    </td>
    </tr>
    <tr valign="top">
    <td>Kelas</td>
    <td>:</td>
    <td>
      <input type="radio" name="kelas" value="VIP">
      VIP<br>

      <input type="radio" name="kelas" value="Festival">Festival
    </td>
  </tr>
  </table>
</form>

```



```

</tr>
<tr>
<td>Jumlah Tiket</td>
<td>:</td>

<td><input type="text" name="jmltiket" placeh
older="Jumlah Beli">
</td>
</tr>
<tr>
<td colspan="3" align="center"><hr>
<input type="submit" value="SIMPAN DATA">
<input type="reset" value="BATAL"><hr>
</td>
</tr>
</table>
</form>
</body>
</html>

```

Hasil.balde.php

```

<!DOCTYPE html>
<html>
<head>
<title>Data Pembelian Tiket</title>
</head>
<body>
<table bgcolor="#EEE8AA" border="0" align="ce
nter" width="40%">
<tr>

<td colspan="3" align="center"><h2>DATA PEMBE
LIAN TIKET KONSER AMAL</h2><hr></td>
</tr>

<tr>
<td>Nama Pemesan</td>
<td>:</td>
<td>{{ $nama }}</td>
</tr>

<tr valign="top">
<td>Kode Studio</td>

```



```

<td>:</td>
<td>{{ $studio }}</td>
</tr>

<tr valign="top">
<td>Bintang Tamu</td>
<td>:</td>
<td>{{ $bt }}</td>
</tr>

<tr valign="top">
<td>Jenis Kelas</td>
<td>:</td>
<td>{{ $kelas }}</td>
</tr>

<tr valign="top">
<td>Harga</td>
<td>:</td>
<td>Rp. {{ $harga }} </td>
</tr>

<tr>
<td>Jumlah Tiket</td>
<td>:</td>
<td>{{ $jmltiket }}</td>
</tr>

<tr><td colspan="3"><hr></td></tr>

<tr>
<td>Total Harga</td>
<td>:</td>
<td>Rp. {{ $total }}</td>
</tr>

<tr><td colspan="3"><hr></td></tr>

<tr><td colspan="3" align="center"><a href="booking">Kembali</a></td>
</tr>

```



```

</table>
</form>
</body>
</html>

```

Langkah berikutnya adalah menambahkan kode di *route* (**web.php**) dan masukkan kode berikut ini.

Bagian atas

```
use App\Http\Controllers\BookingController;
```

Bagian isi

```

Route::get('booking', [BookingController::cla
ss, 'booking']);
Route::post('hasil', [BookingController::clas
s, 'hasil']);

```

Berikut hasil keseluruhan kode di **web.php**

```

<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\MhsController;
use App\Http\Controllers\MahasiswaController;
use App\Http\Controllers\BlogController;
use App\Http\Controllers\LatihanController;
use App\Http\Controllers\BelajarController;
use App\Http\Controllers\BookingController;

/*
|-----
|
| Web Routes
|-----
|
|

```



```

| Here is where you can register web routes f
or your application. These
| routes are loaded by the RouteServiceProvider
er within a group which
| contains the "web" middleware group. Now cr
eate something great!
|
*/

Route::get('/', function () {
    return view('welcome');
});

Route::get('blogi', function(){
    return view('blog');
});

Route::get('mhs', [MhsController::class, 'ind
ex']);
Route::get('mahasiswa', [MahasiswaController:
:class, 'index']);
//Route::get('biodata', [MahasiswaController:
:class, 'biodata']);
Route::get('mhs/{nama}', [MhsController::class
, 'getNama']);

Route::get('biodata', [MhsController::class,
'biodata']);
Route::post('biodata/proses', [MhsController::
class, 'proses']);

Route::get('blog', [BlogController::class, 'i
ndex']);
Route::get('tentang', [BlogController::class,
'tentang']);
Route::get('kontak', [BlogController::class,
'kontak']);
Route::get('matematika', [LatihanController::c
lass, 'index']);
Route::get('pangkat', [LatihanController::clas
s, 'pangkat']);

```



```
Route::get('formulir', [BelajarController::class, 'index']);
Route::get('formulir/hasil', [BelajarController::class, 'hasil']);
```

```
Route::get('booking', [BookingController::class, 'booking']);
Route::post('hasil', [BookingController::class, 'hasil']);
```

Langkah terakhir adalah melihat hasil yang kita buat dari kode-kode program diatas dengan membuka browser dan ketikkan *link* berikut, tapi jangan lupa di aktifkan dulu ya *projectnya* dengan ketikkan kode php artisan **serve**, kemudian jalankan *link* berikut ini. <http://127.0.0.1:8000/booking>

PEMBELIAN TIKET KONSER AMAL

Nama	: Bang Raje
Studio	: Studio 2
Kelas	: ☒ VIP ☐ Festival
Jumlah Tiket	: 5

DATA PEMBELIAN TIKET KONSER AMAL	
Nama Pemesan	: Bang Raje
Kode Studio	: Studio 2
Bintang Tamu	: Raihan
Jenis Kelas	: VIP
Harga	: Rp. 500000
Jumlah Tiket	: 5
Total Harga : Rp. 2500000	
Kembali	



B. Latihan

Form input

Data Pemesanan Tiket

Masukkan Nama Penumpang :	
Alamat Penumpang :	
Kota Tujuan :	Solo
Jumlah Beli :	
Pesan Batal	

Form output

Data Pemesanan Tiket

Nama Penumpang	: yudhistira
Alamat Penumpang	: Jl. Daan Mogot No.1
Kota Tujuan	: Solo
Harga	: 450000
Diskon	: 45000
Total Harga	: 2205000
-Kembali-	

Ketentuan Soal:

1. Ketentuan untuk kota tujuan dan harganya:
 - a. Jika Solo, maka harga Rp 450000
 - b. Jika Semarang, maka harga Rp 350000,
 - c. Jika Yogyakarta, maka harga Rp 400000
 - d. Jika Surabaya, maka harga Rp 550000
2. Total Harga: Harga tiket x Jumlah Beli
3. Ketentuan untuk jumlah beli dan diskon:



- a. Jika jumlah beli lebih dari sama dengan 3, maka diskon 10% dari Total Harga.
- b. Jika kurang dari 3, maka tidak dapat diskon.
4. Total Bayar: Total Harga – diskon
5. Jika klik **Kembali**, maka akan kembali ke halaman input.

Selamat Mencoba!



BAB 13



MEMBUAT CRUD (CREATE READ UPDATE DELETE)

Pada bagian ini kita akan membahas bagaimana membuat CRUD (Create, Read, Update dan Delete) di Laravel. **Create** berfungsi untuk membuat proses inputan data ke *database*, **Read** berfungsi untuk menampilkan data dari *database*, **Update** berfungsi untuk mengubah data pada *database*, sedangkan **Delete** berfungsi untuk menghapus data dari *database*.

A. Membuat CRUD dengan *Query Builder*

Langkah awal untuk membuat CRUD yaitu dengan menggunakan *query builder* dari Laravel. *Query builder* merupakan fitur untuk menjalankan *query database*. Jadi dalam hal ini Laravel sudah menyediakan fungsi-fungsi untuk menjalankan *query database*.

B. Pengaturan *Database*

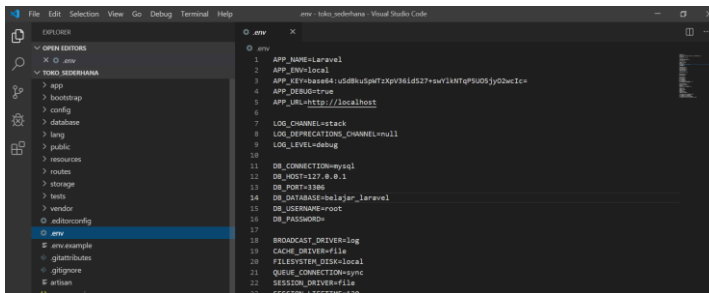
Contoh CRUD yang akan kita buat sebagai contoh yaitu data karyawan. Di sini kita akan membuat beberapa fungsi yaitu sebagai berikut.

1. Menampilkan data dari *database* (Read),
2. Menginput data ke *database* (Create),



3. Mengubah data pada *database*,
4. Menghapus data pada *database*.

Karena *project* kita menggunakan *database*, maka kita harus melakukan konfigurasi *database* terlebih dahulu di *project* Laravel. Untuk mengatur *database* pada Laravel kita akan mengaturnya pada *file .env* letaknya di direktori paling luar. Untuk nama *project* Laravel kita beri nama **toko_sederhana**.



```
1 APP_NAME=Laravel
2 APP_ENV=local
3 APP_KEY=base64:uSdhkUpH7zxpV8Id27+awVikTGP5U03jyQ2wIc=
4 APP_DEBUG=true
5 APP_URL=http://localhost
6
7 LOG_CHANNEL=stack
8 LOG_REPLICATION_CHANNEL=null
9 LOG_LEVEL=debug
10
11 DB_CONNECTION=mysql
12 DB_HOST=127.0.0.1
13 DB_PORT=3306
14 DB_DATABASE=belajar_laravel
15 DB_USERNAME=root
16 DB_PASSWORD=
17
18 BROADCAST_DRIVER=log
19 CACHE_DRIVER=file
20 FILESYSTEM_DISK=local
21 QUEUE_CONNECTION=sync
22 SESSION_DRIVER=file
23 SESSION_LIFETIME=42000
```

Perhatikan kode berikut ini yang ada pada *file .env* *project* Laravel yang kita buat.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=belajar_Laravel
DB_USERNAME=root
DB_PASSWORD=
```

Pada baris **DB_DATABASE=belajar_Laravel** adalah bagian baris untuk menambahkan nama *database* di *project* Laravel yang kita buat, **belajar_Laravel** merupakan nama *database* yang nanti akan kita buat.



C. Persiapan *Database* dan Tabel Karyawan

Di sini tidak di bahas bagaimana cara membuat *database*, silakan baca tutorial membuat *database* di blog-blog maupun di video Channel Youtube. Buat *database* dengan nama **belajar_Laravel** dan buat sebuah tabel dengan nama karyawan. Berikut *filed* dari tabel karyawan yang akan kita buat.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	karyawan_id	int(11)		No	None		AUTO_INCREMENT	Change Drop More
☐	2	nama	varchar(50)	utf8mb4_general_ci	No	None			Change Drop More
☐	3	jabatan	varchar(20)	utf8mb4_general_ci	No	None			Change Drop More
☐	4	umur	int(11)		No	None			Change Drop More
☐	5	alamat	text	utf8mb4_general_ci	No	None			Change Drop More

Jika sudah silakan isi secara manual terlebih dahulu tabel yang sudah kita buat. Misalnya seperti berikut ini.

		karyawan_id	nama	jabatan	umur	alamat
☐	Edit Copy Delete	1	Sabaruddin	Direktur	21	Pontianak-Indonesia
☐	Edit Copy Delete	2	Pak Rajeee	Wakil Direktur	20	Kubu Raya- Indonesia
☐	Edit Copy Delete	3	bang raje	Direktur	21	Pontianak-Indonesia
☐	Edit Copy Delete	6	Sri Murni	Sekretaris	22	Subi-Indonesia
☐	Edit Copy Delete	7	Qona'ah	Karyawan Kasir	21	Subi-Indonesia
☐	Edit Copy Delete	8	Anggelika	Resepsionis	15	Subi-Indonesia

D. Menampilkan Data dari *Database* (Read)

Setelah kita memiliki beberapa data pada tabel karyawan tadi, maka Langkah awal yang akan kita lakukan adalah membuat *route* baru. Berikut kode *route* yang akan kita buat.

```
Route::get('karyawan', [KaryawanController::class, 'index']);
```



Karena kita akan membuat `KaryawanController` maka kita wajib menambahkan kode berikut di halaman **web.php**

```
use App\Http\Controllers\KaryawanController;  
Route yang kita buat di atas merupakan perintah  
untuk menjalankan method index() pada controller  
KaryawanController pada saat route karyawan di akses.
```

Karena kita belum membuat `KaryawanController`, silakan buat terlebih dahulu controller tersebut dengan perintah php artisan.

```
Php artisan make:controller  
KaryawanController
```

Dan karena kita akan menggunakan *query builder* Laravel, maka wajib kita tambahkan kode berikut ini pada halaman **KaryawanController.php**

```
use Illuminate\Support\Facades\DB; //pengguna  
an query builder
```

hasil keseluruhannya sebagai berikut.

```
<?php  
  
namespace App\Http\Controllers;  
  
use Illuminate\Http\Request;  
use Illuminate\Support\Facades\DB; //pengguna  
an query builder  
  
class KaryawanController extends Controller  
{  
    //  
}
```



```
}
```

Berikutnya kita akan membuat method **index()** pada **KaryawanController.php**, berikut kode programnya.

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB; //penggunaan query builder

class KaryawanController extends Controller
{
    //halaman index di view
    public function index()
    {
        //fungsi mengambil data dari tabel karyawan
        $karyawan = DB::table('karyawan')->get();

        //mengirim data karyawan ke bagian view

        return view('index', ['karyawan'=>$karyawan]);
    }
}
```

Perhatikan pada fungsi **index()** yang kita buat dalam controller **KaryawanController.php** di atas, di sana kita mengambil data dari tabel karyawan dengan perintah yang sangat pendek yaitu.

```
$karyawan = DB::table('karyawan')->get();
```



Dengan fungsi **table()** kita menentukan nama tabel yang ingin dipilih. Fungsi **ger()** berguna untuk menampilkan data dari tabel yang kita pilih.

Langkah berikutnya yaitu membuat *file* di bagian view dengan nama **index.blade.php** dan masukkan kode berikut ini.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-
width, initial-scale=1.0">
  <meta http-equiv="X-UA-
Compatible" content="ie=edge">

  <!-- css tabel -->
  <style>
    table {
      border-collapse: collapse;
    }
    {
      width: 100%;
    }
    th
    { height: 50px; }

    b {
      background-color:#228B22; /* warna */
      border: none;
      color: white;
      padding: 15px 32px;
      text-align: center;
      text-decoration: none;
      display: inline-block;
      font-size: 16px;
      box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px
x 20px 0 rgba(0,0,0,0.19);
    }
  </style>
</head>
<body>
```




```

c {
background-color:#FF8C00; /* warna */
border: none;
color: white;
padding: 15px 32px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 16px;
box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px 2
x 20px 0 rgba(0,0,0,0.19);
}

d {
background-color:#B22222; /* warna */
border: none;
color: white;
padding: 15px 32px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 16px;
box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px 2
0px 0 rgba(0,0,0,0.19);
}

</style>
<title>Toko Sederhana</title>
</head>
<body align="center">

<table align="center" width=40%>
<tr>

<td><h2>Data Karyawan Toko Sederhana</h2><
/td>
</tr>
<tr>

```



```

        <td>Jl Natuna, Kawasan Subi, Indonesia | N
        o AAA 8 | HP : 2876 78567 8756
        <hr></td>
    </tr>
    <tr>

        <td><a href="/karyawan/tambah"><b align="c
        enter">+Tambah Data</b></a></td>
    </tr>
</table>
<br>

<table border=1 align="center">
<tr>
    <th>Nama</th>
    <th>Jabatan</th>
    <th>Umur</th>
    <th>Alamat</th>
    <th>Opsional</th>
</tr>
@foreach($karyawan as $p)
<tr>
<td>{{ $p->nama }}</td>
<td>{{ $p->jabatan }}</td>
<td>{{ $p->umur }}</td>
<td>{{ $p->alamat }}</td>
<td>
        <a href="/karyawan/edit/{{ $p-
karyawan_id }}"><c>Edit</c></a>
        |
        <a href="/karyawan/hapus/{{ $p-
karyawan_id }}"><d>Hapus</d></a>
    </td>
</tr>
@endforeach
</table>
</body>
</html>

```

Coba kita perhatikan kode **index.blade.php** di atas. Sebelumnya, pada **KaryawanController** data yang



kita ambil dari tabel karyawan kita simpan dalam **\$karyawan** kemudian kita *passing view* data tersebut di halaman view **index.blade.php**. pada proses ini kita menggunakan perintah kode **foreach()**.

```
@foreach($karyawan as $p)
<tr>
    <td>{{ $p->nama }}</td>
    <td>{{ $p->jabatan }}</td>
    <td>{{ $p->umur }}</td>
    <td>{{ $p->alamat }}</td>
    <td>
        <a href="/karyawan/edit/{{ $p-
        >karyawan_id }}"><c>Edit</c></a>
        |
        <a href="/karyawan/hapus/{{ $p-
        >karyawan_id }}"><d>Hapus</d></a>
    </td>
</tr>
@endforeach
```

Dan sekarang kita akan melihat hasilnya dengan mengakses *link* <http://127.0.0.1:8000/karyawan> berikut hasilnya.



Data Karyawan Toko Sederhana

Jl Natuna, Kawasan Subi, Indonesia | No AAA 8 | HP : 2876 78567 8756

+Tambah Data

Nama	Jabatan	Umur	Alamat	Opsi	
Sabaruddin	Direktur	21	Pontianak-Indonesia	Edit	Hapus
Pak Rajee	Wakil Direktur	20	Kubu Raya- Indonesia	Edit	Hapus
bang raje	Direktur	21	Pontianak-Indonesia	Edit	Hapus
Sri Murni	Sekretaris	22	Subi-Indonesia	Edit	Hapus
Qona'ah	Karyawan Kasir	21	Subi-Indonesia	Edit	Hapus
Angelika	Resepsionis	15	Subi-Indonesia	Edit	Hapus

E. Menginput Data ke *Database* (Create)

Kita langsung saja, pertama dan utama kita harus membuat *route* 'karyawan/tambah' pada *file web.php*. berikut kodenya programnya.

```
Route::get('karyawan/tambah', [KaryawanController::class, 'tambah']);
```

Di sini kita sudah membuat *route* **'/karyawan/tambah'**, maka Langkah berikutnya adalah membuat perintah menjalankan method tambah di **KaryawanController.php**. berikut kode programnya.

```
//halaman menampilkan tambah di view
public function tambah()
{
    return view('tambah');
}
```



Jika sudah Langkah berikutnya yaitu membuat *file tambah.blade.php* pada view, dan berikut kode programnya.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Toko Sederhana</title>
</head>
<body align="center">
<h3>Input Data Karyawan</h3>

<br><br>

<form action="/karyawan/store" method="post">
{{ csrf_field() }}
<table align="center">

    <tr>
        <td>Nama</td>
        <td>:</td>

        <td><input type="text" name="nama" required="required"></td>
    </tr>

    <tr>
        <td>Jabatan</td>
        <td>:</td>

        <td><input type="text" name="jabatan" required="required"></td>
    </tr>

    <tr>
```



```

        <td>Umur</td>
        <td>:</td>

        <td><input type="number" name="umur" required="required"></td>
    </tr>

    <tr>
        <td>Alamat</td>
        <td>:</td>

        <td><textarea name="alamat" required="required"></textarea></td>
    </tr>

    <tr>
        <td><a href="/karyawan">Kembali</a></td>
        <td>|</td>

        <td><input type="submit" value="Simpan Data"></td>
    </tr>
</table>
</form>

</body>
</html>

```

Perhatikan lagi pada view **tambah.blade.php** yang kita buat di atas, untuk action *form*-nya kita buat diarahkan ke route '**karyawan/store**'. Karena kita ingin nanti route '**karyawan/store**' yang berfungsi pemrosesan data yang di input agar bisa diolah oleh controller. Tambahan di sini kita menemukan kode **{{ csrf_field() }}**, yaitu **csrf protection**, sejenis fitur keamanan untuk pencegahan penginputan data dari luar aplikasi atau sistem kita.



Karena pada *form* tambah karyawan action-nya kita buat `'/karyawan/store'` maka *form submit* akan di arahkan ke *route* `'/karyawan/store'` untuk di proses.

Sekarang kita buat *route* baru lagi, untuk *route* `'/karyawan/store'`. Berikut kode programnya di **web.php**

```
Route::post('/karyawan/store', [KaryawanContro  
ller::class, 'store']);
```

Pada *route* ini, kita tidak menggunakan method **get** melainkan method **post**, karena untuk mengirimkan data melalui *form* ke *route* `'/karyawan/store'`. Sekarang kita akan menambahkan method **store** di *file* **KaryawanController.php** berikut kode programnya.

```
//methode untuk insert data ke tabel karyawan  
public function store(Request $request)  
{  
    //insert data ke tabel karyawan  
    DB::table('karyawan')->insert([  
        'nama' => $request->nama,  
        'jabatan' => $request->jabatan,  
        'umur' => $request->umur,  
        'alamat' => $request->alamat  
    ]);  
    //kembali ke halaman karyawan  
    return redirect('/karyawan');  
}
```

Jika sudah mari kita coba untuk proses inputannya dengan mengisi beberapa data saja. Silakan klik tombol **+tambah** data pada *link* <http://127.0.0.1:8000/karyawan> berikut hasilnya.



Data Karyawan Toko Sederhana

Jl Natuna, Kawasan Subi, Indonesia | No AAA 8 | HP : 2876 78567 8756

+Tambah Data

Nama	Jabatan	Umur	Alamat	Opsi	
Sabaruddin	Direktur	21	Pontianak-Indonesia	Edit	Hapus
Pak Rajee	Wakil Direktur	20	Kubu Raya- Indonesia	Edit	Hapus
bang raje	Direktur	21	Pontianak-Indonesia	Edit	Hapus
Sri Murni	Sekretaris	22	Subi-Indonesia	Edit	Hapus
Qona'ah	Karyawan Kasir	21	Subi-Indonesia	Edit	Hapus
Angelika	Resepsionis	15	Subi-Indonesia	Edit	Hapus
Abd Aziz	Koordinator AA	50	Terayak-Indonesia	Edit	Hapus

Data dengan nama Abd Aziz berhasil ditambahkan dan tersimpan di *database*.

				karyawan_id	nama	jabatan	umur	alamat
☐	 Edit	 Copy	 Delete	1	Sabaruddin	Direktur	21	Pontianak-Indonesia
☐	 Edit	 Copy	 Delete	2	Pak Rajee	Wakil Direktur	20	Kubu Raya- Indonesia
☐	 Edit	 Copy	 Delete	3	bang raje	Direktur	21	Pontianak-Indonesia
☐	 Edit	 Copy	 Delete	6	Sri Murni	Sekretaris	22	Subi-Indonesia
☐	 Edit	 Copy	 Delete	7	Qona'ah	Karyawan Kasir	21	Subi-Indonesia
☐	 Edit	 Copy	 Delete	8	Anggelika	Resepsionis	15	Subi-Indonesia
☐	 Edit	 Copy	 Delete	9	Abd Aziz	Koordinator AA	50	Terayak-Indonesia

F. Update Data Pada *Database* (Update)

Coba kita perhatikan lagi pada view yang menampilkan data karyawan yang kita buat sebelumnya, yaitu **index.blade.php** di sana kita membuat tombol edit berikut kode programnya.

```
<a href="/karyawan/edit/{{ $p-  
>karyawan_id }}"><c>Edit</c></a>
```



Pada tombol tersebut kita memerintahkan untuk mengalihkan halaman ke *route* **‘/karyawan/edit’** sambil mengirimkan data id data yang akan di edit.

Sekarang kita buat *route* baru untuk **‘/karyawan/edit’** seperti berikut.

```
Route::get('/karyawan/edit/{id}', [KaryawanController::class, 'edit']);
```

Bisa kita lihat pada *route* ini, data id yang dikirim pada url kita istilahkan dengan **{id}** dan akan kita perintahkan untuk menjalankan method **edit** pada **KaryawanController.php**.

Berikut kita akan membuat method **edit** di **KaryawanController.php**, berikut kode programnya.

```
//method untuk edit data karyawan
public function edit($id)
{
    //mengambil data berdasarkan id yang dipilih
    $karyawan = DB::table('karyawan')-
    >where('karyawan_id', $id)->get();

    //passing data karyawan di dapat ke view edit
    .balade.php

    return view('edit', ['karyawan' => $karyawan]);
}
```

Data id yang dikirim dari *route* tadi kita tangkap dalam parameter metode edit (**edit(\$id)**). Selanjutnya kita ambil data pegawai dari *database* dengan menggunakan **query builder** berikut.



```

/mengambil data berdasarkan id yang dipilih
$karyawan = DB::table('karyawan')-
>where('karyawan_id',$id)->get();

```

Berikutnya, kita akan membuat view baru dengan nama **edit.blade.php** karena kita akan menampilkan data karyawan yang ingin di edit tadi di dalam *form* edit. Berikut kode program **edit.blade.php**

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-
width, initial-scale=1.0">
    <meta http-equiv="X-UA-
Compatible" content="ie=edge">
    <title>Toko Sederhana</title>
</head>
<body>
@foreach($karyawan as $p)
<form action="/karyawan/update" method="post"
>
    {{ csrf_field() }}
    <table align="center">
        <tr>

            <td colspan=3><input type="hidden" name="i
            d" value="{{ $p->karyawan_id }}"></td>
        </tr>

        <tr>
            <td>Nama</td>
            <td>:</td>

            <td><input type="text" required="required"
            name="nama" value="{{ $p->nama }}"></td>
        </tr>

        <tr>

```



```

        <td>Jabatan</td>
        <td>:</td>

        <td><input type="text" required="required"
            name="jabatan" value="{{ $p-
            >jabatan }}"></td>
    </tr>

    <tr>
        <td>Umur</td>
        <td>:</td>

        <td><input type="number" required="require
            d" name="umur" value="{{ $p-
            >umur }}"></td>
    </tr>

    <tr>
        <td>Alamat</td>
        <td>:</td>

        <td><textarea required="required" name="al
            amat">{{ $p->alamat }}</textarea></td>
    </tr>

    <tr>
        <td><a href="/karyawan">Kembali</a></td>
        <td>|</td>

        <td><input type="submit" value="Update"></
            td>
    </tr>
</table>
</form>
@endforeach
</body>
</html>

```

Karena *form* edit *form* actionnya **'/karyawan/update'** dengan method **post()**, maka kita



akan menambahkan lagi route **'/karyawan/update'** pada **web.php** berikut kode programnya.

```
Route::post('/karyawan/update', [KaryawanController::class, 'update']);
```

Langkah berikutnya yaitu menambahkan method **update** pada **KaryawanController.php** seperti berikut ini.

```
//update data karyawan
public function update(Request $request)
{
    //update data pegawai
    DB::table('karyawan')-
>where('karyawan_id',$request->id)->update([
        'nama' => $request->nama,
        'jabatan' =>$request->jabatan,
        'umur' => $request->umur,
        'alamat' => $request->alamat
    ]);

    //kembali ke halaman karyawan
    return redirect('/karyawan');
}
```

Sekarang kita akan mencoba hasil dari kode program yang kita buat dengan mengeklik tombol **edit** melalui laman <http://127.0.0.1:8000/karyawan>



Nama : Sabaruddin

Jabatan : Direktur

Umur : 21

Alamat : Pontianak-Indonesia

[Kembali](#) |

Kita akan meng-*update* nama Sabaruddin menjadi Raja Sabaruddin dan Umur 21 menjadi 25.

Data Karyawan Toko Sederhana

Jl Natuna, Kawasan Subi, Indonesia | No AAA 8 | HP : 2876 78567 8756

Nama	Jabatan	Umur	Alamat	Ops	
Raja Sabaruddin	Direktur	25	Pontianak-Indonesia	Edit	Hapus
Pak Rajeee	Wakil Direktur	20	Kubu Raya- Indonesia	Edit	Hapus
bang raje	Direktur	21	Pontianak-Indonesia	Edit	Hapus
Sri Murni	Sekretaris	22	Subi-Indonesia	Edit	Hapus
Qona'ah	Karyawan Kasir	21	Subi-Indonesia	Edit	Hapus
Angelika	Resepsionis	15	Subi-Indonesia	Edit	Hapus
Abd Aziz	Koordinator AA	50	Terayak-Indonesia	Edit	Hapus

G. Menghapus Data dari *Database* (Delete)

Sama seperti halnya edit, kita sudah membuat kode program hapus di **index.blade.php**. maka kita harus menambahkan juga *route* **'/karyawan/hapus'** pada *route* atau pada *file* web.php seperti berikut ini.



```
Route::get('/karyawan/hapus/{id}', [KaryawanC  
ontroller::class, 'hapus']);
```

Setelah membuat *route* maka Langkah berikutnya adalah membuat method **hapus()** pada **KaryawanController.php**. berikut kode programnya.

```
//method untuk hapus data karyawan  
public function hapus($id)  
{  
  
//menghapus data karyawan berdasarkan id yang  
di pilih  
DB::table('karyawan')-  
>where('karyawan_id',$id)->delete();  
  
//kembali ke halaman karyawan  
return redirect('/karyawan');  
}
```

Sekarang kita kan melihat hasilnya dengan mengeklik tombol **hapus** pada *link* berikut ini <http://127.0.0.1:8000/karyawan> pada kasus ini kita akan menghapus nama **Abd Aziz** dari *link* tersebut.



Data Karyawan Toko Sederhana

Jl Natuna, Kawasan Subi, Indonesia | No AAA 8 | HP : 2876 78567 8756

+Tambah Data

Nama	Jabatan	Umur	Alamat	Opsi	
Raja Sabaruddin	Direktur	25	Pontianak-Indonesia	Edit	Hapus
Pak Rajee	Wakil Direktur	20	Kubu Raya- Indonesia	Edit	Hapus
bang raje	Direktur	21	Pontianak-Indonesia	Edit	Hapus
Sri Murni	Sekretaris	22	Subi-Indonesia	Edit	Hapus
Qona'ah	Karyawan Kasir	21	Subi-Indonesia	Edit	Hapus
Anggelika	Resepsionis	15	Subi-Indonesia	Edit	Hapus

Data berhasil dihapus dan ditabel karyawan data **Abd Aziz** juga sudah tidak ada lagi. Berikut hasilnya.

				karyawan_id	nama	jabatan	umur	alamat
☐	 Edit	 Copy	 Delete	1	Raja Sabaruddin	Direktur	25	Pontianak-Indonesia
☐	 Edit	 Copy	 Delete	2	Pak Rajee	Wakil Direktur	20	Kubu Raya- Indonesia
☐	 Edit	 Copy	 Delete	3	bang raje	Direktur	21	Pontianak-Indonesia
☐	 Edit	 Copy	 Delete	6	Sri Murni	Sekretaris	22	Subi-Indonesia
☐	 Edit	 Copy	 Delete	7	Qona'ah	Karyawan Kasir	21	Subi-Indonesia
☐	 Edit	 Copy	 Delete	8	Anggelika	Resepsionis	15	Subi-Indonesia

H. Kode Lengkap Toko_Sederhana Toko_sederhana/routes/web.php

```
<?php
```

```
use Illuminate\Support\Facades\Route;
```

```
use App\Http\Controllers\KaryawanController;
```



```

/*
|-----
|
| Web Routes
|-----
|
| Here is where you can register web routes f
or your application. These
| routes are loaded by the RouteServiceProvider
er within a group which
| contains the "web" middleware group. Now cr
eate something great!
|
*/

Route::get('/', function () {
    return view('welcome');
});

Route::get('karyawan', [KaryawanController::cl
ass, 'index']);
Route::get('karyawan/tambah', [KaryawanControl
ler::class, 'tambah']);
Route::post('/karyawan/store', [KaryawanContro
ller::class, 'store']);
Route::get('/karyawan/edit/{id}', [KaryawanCo
ntroller::class, 'edit']);
Route::post('/karyawan/update', [KaryawanCont
roller::class, 'update']);
Route::get('/karyawan/hapus/{id}', [KaryawanC
ontroller::class, 'hapus']);

```

Toko_sederhana/App/Http/Controllers/KaryawanCo ntroller.php

```

<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

```




```
use Illuminate\Support\Facades\DB; //pengguna  
an query builder
```

```
class KaryawanController extends Controller  
{  
    //halaman index di view  
    public function index()  
    {  
        //fungsi mengambil data dari tabel karyawan  
        $karyawan = DB::table('karyawan')->get();  
  
        //mengirim data karyawan ke bagian view  
  
        return view('index', ['karyawan'=>$karyawan])  
    ;  
    }  
  
    //halaman menampilkan tambah di view  
    public function tambah()  
    {  
        return view('tambah');  
    }  
  
    //methode  
    untuk insert data ke tabel karyawan  
    public function store(Request $request)  
    {  
        //insert data ke tabel karyawan  
        DB::table('karyawan')->insert([  
            'nama' => $request->nama,  
            'jabatan' => $request->jabatan,  
            'umur' => $request->umur,  
            'alamat' => $request->alamat  
        ]);  
        //kembali ke halaman karyawan  
        return redirect('/karyawan');  
    }  
  
    //method untuk edit data karyawan  
    public function edit($id)  
    {  
        //mengambil data berdasarkan id yang dipilih
```



```

    $karyawan = DB::table('karyawan')-
>where('karyawan_id',$id)->get();

//passing data karyawan di dapat ke view edit
.balade.php

return view('edit',['karyawan' => $karyawan])
;
}

//update data karyawan
public function update(Request $request)
{
    //update data pegawai
    DB::table('karyawan')-
>where('karyawan_id',$request->id)->update([
    'nama' => $request->nama,
    'jabatan' =>$request->jabatan,
    'umur' => $request->umur,
    'alamat' => $request->alamat
]);

//kembali ke halaman karyawan
return redirect('/karyawan');
}

//method untuk hapus data karyawan
public function hapus($id)
{

//menghapus data karyawan berdasarkan id yang
di pilih
    DB::table('karyawan')-
>where('karyawan_id',$id)->delete();

    //kembali ke halaman karyawan
    return redirect('/karyawan');
}
}

```



Toko_sederhana/resources/view/index.blade.php

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">

<!-- css tabel -->
<style>
    table {
        border-collapse: collapse;
    }
    {
        width: 100%;
    }
    th
    { height: 50px; }

    b {
        background-color:#228B22; /* warna */
        border: none;
        color: white;
        padding: 15px 32px;
        text-align: center;
        text-decoration: none;
        display: inline-block;
        font-size: 16px;
        box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6
px 20px 0 rgba(0,0,0,0.19);
    }

    c {
        background-color:#FF8C00; /* warna */
        border: none;
        color: white;
```



```

padding: 15px 32px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 16px;
box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6
px 20px 0 rgba(0,0,0,0.19);
}

d {
background-color:#B22222; /* warna */
border: none;
color: white;
padding: 15px 32px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 16px;
box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6
px 20px 0 rgba(0,0,0,0.19);
}

</style>
<title>Toko Sederhana</title>
</head>
<body align="center">

    <table align="center" width=40%>
    <tr>

    <td><h2>Data Karyawan Toko Sederhana</h2><
    /td>
    </tr>
    <tr>

    <td>Jl Natuna, Kawasan Subi, Indonesia | N
    o AAA 8 | HP : 2876 78567 8756
    <hr></td>
    </tr>
    <tr>

```



```

<td><a href="/karyawan/tambah"><b align="center">+Tambah Data</b></a></td>
</tr>
</table>
<br>

<table border=1 align="center">
<tr>
<th>Nama</th>
<th>Jabatan</th>
<th>Umur</th>
<th>Alamat</th>
<th>Ops</th>
</tr>
@foreach($karyawan as $p)
<tr>
<td>{{ $p->nama }}</td>
<td>{{ $p->jabatan }}</td>
<td>{{ $p->umur }}</td>
<td>{{ $p->alamat }}</td>
<td>
<a href="/karyawan/edit/{{ $p-
>karyawan_id }}"><c>Edit</c></a>
|
<a href="/karyawan/hapus/{{ $p-
>karyawan_id }}"><d>Hapus</d></a>
</td>
</tr>
@endforeach
</table>
</body>
</html>

```

Toko_sederhana/resources/view/tambah.blade.php

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">

```



```

        <meta name="viewport" content="width=device
        e-width, initial-scale=1.0">
        <meta http-equiv="X-UA-
        Compatible" content="ie=edge">
        <title>Toko Sederhana</title>
    </head>
    <body align="center">
    <h3>Input Data Karyawan</h3>

    <br><br>

    <form action="/karyawan/store" method="post">
    {{ csrf_field() }}
    <table align="center">

        <tr>
            <td>Nama</td>
            <td>:</td>

            <td><input type="text" name="nama" require
            d="required"></td>
        </tr>

        <tr>
            <td>Jabatan</td>
            <td>:</td>

            <td><input type="text" name="jabatan" requi
            red="required"></td>
        </tr>

        <tr>
            <td>Umur</td>
            <td>:</td>

            <td><input type="number" name="umur" requi
            red="required"></td>
        </tr>

        <tr>
            <td>Alamat</td>

```



```

        <td>:</td>

        <td><textarea name="alamat" required="required"></textarea></td>
    </tr>

    <tr>
        <td><a href="/karyawan">Kembali</a></td>
        <td>|</td>

        <td><input type="submit" value="Simpan Data"></td>
    </tr>
</table>
</form>

</body>
</html>

```

Toko_sederhana/resources/view/edit.blade.php

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Toko Sederhana</title>
</head>
<body>
    @foreach($karyawan as $p)
    <form action="/karyawan/update" method="post"
    >
        {{ csrf_field() }}
    <table align="center">
        <tr>

```



```

        <td colspan=3><input type="hidden" name="
        id" value="{{ $p->karyawan_id }}"></td>
</tr>

<tr>
    <td>Nama</td>
    <td>:</td>

    <td><input type="text" required="required"
        name="nama" value="{{ $p->nama }}"></td>
</tr>

<tr>
    <td>Jabatan</td>
    <td>:</td>

    <td><input type="text" required="required"
        name="jabatan" value="{{ $p-
        >jabatan }}"></td>
</tr>

<tr>
    <td>Umur</td>
    <td>:</td>

    <td><input type="number" required="require
        d" name="umur" value="{{ $p-
        >umur }}"></td>
</tr>

<tr>
    <td>Alamat</td>
    <td>:</td>

    <td><textarea required="required" name="al
        amat">{{ $p->alamat }}</textarea></td>
</tr>

<tr>
    <td><a href="/karyawan">Kembali</a></td>
    <td>|</td>

```




```
        <td><input type="submit" value="Update"></td>
    </tr>
</table>
</form>
@endforeach
</body>
</html>
```

I. Latihan

Silakan buat tabel nama barang pada *database belajar_Laravel* dan buar CRUD dari tabel barang tersebut. Selamat mencoba.





BAB 14



MEMBUAT LAPORAN SEDERHANA DI LARAVEL

Setelah kita membuat perintah CRUD pada bab sebelumnya, maka pada pertemuan ini kita akan membuat cetak laporan untuk melengkapi pembelajaran Laravel kita.

A. Kode Program

Tahap awal untuk membuat laporan yaitu menambahkan satu baris kode pada **index.blade.php** seperti berikut ini.

```
<tr>

    <td><a href="/karyawan/tambah"><b align="center">+Tambah Data</b></a>|<a href="/laporan" target="_blank"><b align="center">Cetak Laporan</b></a></td>

</tr>
```

Perintah kode di atas berfungsi membuat tombol **Cetak Laporan**, jika kita klik tombol tersebut, maka akan mengarahkan ke halaman **laporan.blade.php**. berikut kode lengkapnya di **index.blade.php**

```
<!DOCTYPE html>
<html lang="en">
<head>
```



```

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta http-equiv="X-UA-Compatible" content="ie=edge">

<!-- css tabel -->
<style>
table {
border-collapse: collapse;
}
{
width: 100%;
}
th
{ height: 50px; }

b {
background-color:#228B22; /* warna */
border: none;
color: white;
padding: 15px 32px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 16px;
box-shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px 20px 0 rgba(0,0,0,0.19);
}

c {
background-color:#FF8C00; /* warna */
border: none;
color: white;
padding: 15px 32px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 16px;

```



```

        box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px
x 20px 0 rgba(0,0,0,0.19);
    }

    d {
background-color:#B22222; /* warna */
border: none;
color: white;
padding: 15px 32px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 16px;
        box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px
x 20px 0 rgba(0,0,0,0.19);
    }

</style>
<title>Toko Sederhana</title>
</head>
<body align="center">

    <table align="center" width=40%>
    <tr>

    <td><h2>Data Karyawan Toko Sederhana</h2></td>
    >
    </tr>

    <tr>

    <td><a href="/karyawan/tambah"><b align="center">+Tambah Data</b></a>|<a href="/laporan" target="_blank"><b align="center">Cetak Laporan</b></a></td>

    </tr>
    </table>
<br>

```



```

<table border=1 align="center">
<tr>
    <th>Nama</th>
    <th>Jabatan</th>
    <th>Umur</th>
    <th>Alamat</th>
    <th>Ops</th>
</tr>
@foreach($karyawan as $p)
<tr>
    <td>{{ $p->nama }}</td>
    <td>{{ $p->jabatan }}</td>
    <td>{{ $p->umur }}</td>
    <td>{{ $p->alamat }}</td>
    <td>
        <a href="/karyawan/edit/{{ $p-
        >karyawan_id }}"><c>Edit</c></a>
        |
        <a href="/karyawan/hapus/{{ $p-
        >karyawan_id }}"><d>Hapus</d></a>
    </td>
</tr>
@endforeach
</table>
</body>
</html>

```

Jika sudah selesai makan kita akan menambahkan method **laporan** di **KaryawanContorller.php** berikut kode programnya.

```

public function laporan()
{
    //fungsi mengambil data dari tabel karyawan
    $karyawan = DB::table('karyawan')->get();

    //mengirim data karyawan ke bagian view

    return view('laporan', ['karyawan'=>$karyawan
]);

```



```
}
```

Berikut kode selengkapnya pada *file* **KaryawanController.php**.

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB; //pengguna
an query builder

class KaryawanController extends Controller
{
    //halaman index di view
    public function index()
    {

        //fungsi mengambil data dari tabel karyawan
        $karyawan = DB::table('karyawan')->get();

        //mengirim data karyawan ke bagian view

        return view('index', ['karyawan'=>$karyawan
    ]);
}

//halaman menampilkan tambah di view
public function tambah()
{
    return view('tambah');
}

//methode
untuk insert data ke tabel karyawan
public function store(Request $request)
{
    //insert data ke tabel karyawan
    DB::table('karyawan')->insert([
```



```

        'nama' => $request->nama,
        'jabatan' => $request->jabatan,
        'umur' => $request->umur,
        'alamat' => $request->alamat
    ]);
    //kembali ke halaman karyawan
    return redirect('/karyawan');
}

//method untuk edit data karyawan
public function edit($id)
{
    //mengambil data berdasarkan id yang dipilih
    $karyawan = DB::table('karyawan')-
    >where('karyawan_id',$id)->get();

    //passing data karyawan di dapat ke view edit.balade.php

    return view('edit',['karyawan' => $karyawan
    ]);
}

//update data karyawan
public function update(Request $request)
{
    //update data pegawai
    DB::table('karyawan')-
    >where('karyawan_id',$request->id)-
    >update([
        'nama' => $request->nama,
        'jabatan' => $request->jabatan,
        'umur' => $request->umur,
        'alamat' => $request->alamat
    ]);

    //kembali ke halaman karyawan
    return redirect('/karyawan');
}

```




```

//method untuk hapus data karyawan
public function hapus($id)
{

    //menghapus data karyawan berdasarkan id yang di pilih
    DB::table('karyawan')->where('karyawan_id',$id)->delete();

    //kembali ke halaman karyawan
    return redirect('/karyawan');
}

public function laporan()
{
    //fungsi mengambil data dari tabel karyawan
    $karyawan = DB::table('karyawan')->get();

    //mengirim data karyawan ke bagian view

    return view('laporan', ['karyawan'=>$karyawan]);
}
}

```

Di sini kita mengambil data dari tabel karyawan dan akan di kirimkan ke view yaitu **laporan.blade.php**. karena kita belum memiliki *file* laporan.blade.php maka Langkah selanjutnya adalah membuat *file* dengan nama laporan.blade.php dan masukkan kode berikut ini.

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

```



```

<meta http-equiv="X-UA-
Compatible" content="ie=edge">

<!-- css tabel -->
<style>
    table {
        border-collapse: collapse;
    }
    {
        width: 100%;
    }
    th
    { height: 50px; }

    b {
        background-color:#228B22; /* warna */
        border: none;
        color: white;
        padding: 15px 32px;
        text-align: center;
        text-decoration: none;
        display: inline-block;
        font-size: 16px;
        box-
        shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6
        px 20px 0 rgba(0,0,0,0.19);
    }

    c {
        background-color:#FF8C00; /* warna */
        border: none;
        color: white;
        padding: 15px 32px;
        text-align: center;
        text-decoration: none;
        display: inline-block;
        font-size: 16px;
        box-
        shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6
        px 20px 0 rgba(0,0,0,0.19);
    }

```



```

        d {
            background-color:#B22222; /* warna */
            border: none;
            color: white;
            padding: 15px 32px;
            text-align: center;
            text-decoration: none;
            display: inline-block;
            font-size: 16px;
            box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6
px 20px 0 rgba(0,0,0,0.19);
        }

</style>
<title>Toko Sederhana</title>
</head>
<body align="center">

    <table align="center" width=40%>
    <tr>

<td><h2>Data Karyawan Toko Sederhana</h2></td>
>
    </tr>
    <tr>

<td>Jl Natuna, Kawasan Subi, Indonesia | No A
AA 8 | HP : 2876 78567 8756
    <hr></td>
    </tr>
    <tr>

</table>

<br>

    <table border=1 align="center">
    <tr>
        <th>Nama</th>
        <th>Jabatan</th>
        <th>Umur</th>

```



```

        <th>Alamat</th>
    </tr>
    @foreach($karyawan as $p)
    <tr>
        <td align="left">{{ $p->nama }}</td>
        <td align="left">{{ $p->jabatan }}</td>
        <td align="center">{{ $p->umur }}</td>
        <td align="left">{{ $p->alamat }}</td>

    </tr>
    @endforeach

</table>

<p>Dicetak di Pontianak, <?php echo date('l,
d-m-Y H:i:s'); ?></p>
</body>
</html>

```

Sebelum melihat hasilnya jangan lupa untuk menambahkan *route* baru untuk menampilkan *route* laporan. Berikut kode program *route* pada web.php

```
Route::get('laporan', [KaryawanController::class, 'laporan']);
```

Dan berikut kode program lebih lengkapnya pada *file web.php*

```

<?php

use Illuminate\Support\Facades\Route;

use App\Http\Controllers\KaryawanController;

/*
|-----
|
| Web Routes

```



```

|-----
|
| Here is where you can register web routes f
or your application. These
| routes are loaded by the RouteServiceProvider
within a group which
| contains the "web" middleware group. Now cr
eate something great!
|
*/

Route::get('/', function () {
    return view('welcome');
});

Route::get('karyawan', [KaryawanController::cl
ass, 'index']);
Route::get('karyawan/tambah', [KaryawanControl
ler::class, 'tambah']);
Route::post('/karyawan/store', [KaryawanContro
ller::class, 'store']);
Route::get('/karyawan/edit/{id}', [KaryawanCo
ntroller::class, 'edit']);
Route::post('/karyawan/update', [KaryawanCont
roller::class, 'update']);
Route::get('/karyawan/hapus/{id}', [KaryawanC
ontroller::class, 'hapus']);
Route::get('laporan', [KaryawanController::cla
ss, 'laporan']);

```

Sekarang kita akan melihat hasilnya dengan mengklik [link](http://127.0.0.1:8000/karyawan) berikut ini <http://127.0.0.1:8000/karyawan>



Data Karyawan Toko Sederhana

[+Tambah Data](#)[Cetak Laporan](#)

Nama	Jabatan	Umur	Alamat	Opsi	
Raja Sabaruddin	Direktur	25	Pontianak-Indonesia	Edit	Hapus
Pak Rajeee	Wakil Direktur	20	Kubu Raya- Indonesia	Edit	Hapus
bang raje	Direktur	21	Pontianak-Indonesia	Edit	Hapus
Sri Murni	Sekretaris	22	Subi-Indonesia	Edit	Hapus
Qona'ah	Karyawan Kasir	21	Subi-Indonesia	Edit	Hapus
Anggelika	Resepsionis	15	Subi-Indonesia	Edit	Hapus

Data Karyawan Toko Sederhana

Jl Natuna, Kawasan Subi, Indonesia | No AAA 8 | HP : 2876 78567 8756

Nama	Jabatan	Umur	Alamat
Raja Sabaruddin	Direktur	25	Pontianak-Indonesia
Pak Rajeee	Wakil Direktur	20	Kubu Raya- Indonesia
bang raje	Direktur	21	Pontianak-Indonesia
Sri Murni	Sekretaris	22	Subi-Indonesia
Qona'ah	Karyawan Kasir	21	Subi-Indonesia
Anggelika	Resepsionis	15	Subi-Indonesia

Dicetak di Pontianak, Monday, 17-10-2022 07:52:09

B. Latihan

Buatlah kode program cetak laporan untuk data-data seperti data barang dan lain-lain sebagai Latihan.

Terima Kasih



BAB 15



MEMBUAT PAGINATION DI LARAVEL

A. Definisi *Pagination*

Pagination berfungsi untuk membuat halaman penomoran pada tampilan tabel Laravel. Misalnya kita memiliki banyak *row* misalnya 100 atau lebih *row data*, tentu jika tidak menggunakan *pagination* tabel tersebut akan Panjang ke bawah.

Di sinilah peran penting *pagination* atau penomoran halaman diperlukan, jadi kita bisa mengatur beberapa jumlah *row* yang kita tampilkan.

B. Membuat *Pagination*

Pada proses membuat *pagination* kita masih menggunakan *project* yang sama yaitu *project toko_sederhana*. Jadi kita tidak perlu membuat *file* baru cukup kita modifikasi saja *file* yang sudah ada.

Pertama kita akan mengubah di bagian *file* *KaryawanController.php* yang sebelumnya adalah sebagai berikut.

```
$karyawan = DB::table('karyawan')->get();
```



Kemudian kita ganti dengan fungsi pagination, seperti berikut ini.

```
$karyawan = DB::table('karyawan')->paginate(5);
```

Dan berikut kode program lengkapnya pada *file* **KaryawanController.php**.

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB; //penggunaan query builder

class KaryawanController extends Controller
{
    //halaman index di view
    public function index()
    {
        //fungsi pagination
        $karyawan = DB::table('karyawan')->paginate(5);
        //fungsi mengambil data dari tabel karyawan
        // $karyawan = DB::table('karyawan')->get();

        //mengirim data karyawan ke bagian view

        return view('index', ['karyawan'=>$karyawan]);
    }

    //halaman menampilkan tambah di view
    public function tambah()
    {
        return view('tambah');
    }
}
```




```

//methode
untuk insert data ke tabel karyawan
public function store(Request $request)
{
    //insert data ke tabel karyawan
    DB::table('karyawan')->insert([
        'nama' => $request->nama,
        'jabatan' => $request->jabatan,
        'umur' => $request->umur,
        'alamat' => $request->alamat
    ]);
    //kembali ke halaman karyawan
    return redirect('/karyawan');
}

//method untuk edit data karyawan
public function edit($id)
{
    //mengambil data berdasarkan id yang dipilih
    $karyawan = DB::table('karyawan')-
    >where('karyawan_id',$id)->get();

    //passing data karyawan di dapat ke view edit
    .balade.php

    return view('edit', ['karyawan' => $karyawan])
    ;
}

//update data karyawan
public function update(Request $request)
{
    //update data pegawai
    DB::table('karyawan')-
    >where('karyawan_id',$request->id)->update([
        'nama' => $request->nama,
        'jabatan' => $request->jabatan,
        'umur' => $request->umur,
        'alamat' => $request->alamat
    ]);
}

```



```

//kembali ke halaman karyawan
return redirect('/karyawan');
}

//method untuk hapus data karyawan
public function hapus($id)
{

//menghapus data karyawan berdasarkan id yang
di pilih
DB::table('karyawan')-
>where('karyawan_id',$id)->delete();

//kembali ke halaman karyawan
return redirect('/karyawan');
}

public function laporan()
{

//fungsi mengambil data dari tabel karyawan
$karyawan = DB::table('karyawan')->get();

//mengirim data karyawan ke bagian view

return view('laporan', ['karyawan'=>$karyaw
an]);
}
}

```

Karena kita menggunakan method **index()** dan menampilkan view **index.blade.php**, tentunya kita akan tambahkan sedikit kode program seperti berikut ini.

```

<br/>
<p>Halaman : {{ $karyawan->currentPage() }}
| Jumlah Data : {{ $karyawan-
>total() }} | Data Per Halaman : {{ $karyawan
->perPage() }}</p>
{{ $karyawan->links() }}

```



Berikut kode lengkapnya untuk *file* `index.blade.php`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-
width, initial-scale=1.0">
  <meta http-equiv="X-UA-
Compatible" content="ie=edge">

  <!-- css tabel -->
  <style>
    table {
      border-collapse: collapse;
    }
    {
      width: 100%;
    }
    th
    { height: 50px; }

    b {
      background-color:#228B22; /* warna */
      border: none;
      color: white;
      padding: 15px 32px;
      text-align: center;
      text-decoration: none;
      display: inline-block;
      font-size: 16px;
      box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px
x 20px 0 rgba(0,0,0,0.19);
    }

    c {
      background-color:#FF8C00; /* warna */
      border: none;
      color: white;
      padding: 15px 32px;
```



```

        text-align: center;
        text-decoration: none;
        display: inline-block;
        font-size: 16px;
        box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px
x 20px 0 rgba(0,0,0,0.19);
    }

    d {
        background-color:#B22222; /* warna */
        border: none;
        color: white;
        padding: 15px 32px;
        text-align: center;
        text-decoration: none;
        display: inline-block;
        font-size: 16px;
        box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px
x 20px 0 rgba(0,0,0,0.19);
    }

</style>
<title>Toko Sederhana</title>
</head>
<body align="center">

    <table align="center" width=40%>
    <tr>

    <td><h2>Data Karyawan Toko Sederhana</h2></td>
    </tr>

    <tr>

    <td><a href="/karyawan/tambah"><b align="cent
er">+Tambah Data</b></a>|<a href="/laporan" t
arget="_blank"><b align="center">Cetak Lapora
n</b></a></td>

```



```

</tr>
</table>

<br>

<table border=1 align="center">
<tr>
    <th>Nama</th>
    <th>Jabatan</th>
    <th>Umur</th>
    <th>Alamat</th>
    <th>Ops</th>
</tr>
@foreach($karyawan as $p)
<tr>
    <td>{{ $p->nama }}</td>
    <td>{{ $p->jabatan }}</td>
    <td>{{ $p->umur }}</td>
    <td>{{ $p->alamat }}</td>
    <td>
        <a href="/karyawan/edit/{{ $p-
>karyawan_id }}"><c>Edit</c></a>
        |
        <a href="/karyawan/hapus/{{ $p-
>karyawan_id }}"><d>Hapus</d></a>
    </td>
</tr>
@endforeach

</table>
<br/>
<p>Halaman : {{ $karyawan->currentPage() }}
| Jumlah Data : {{ $karyawan-
>total() }} | Data Per Halaman : {{ $karyawan
->perPage() }}</p>
    {{ $karyawan->links() }}
</body>
</html>

```



Sekarang kita akan melihat hasilnya melalui link berikut ini <http://127.0.0.1:8000/karyawan>



PERINTAH PENCARIAN DI LARAVEL

Form pencarian yang akan kita buat adalah *form* data pencarian data karyawan yang sebelumnya sudah pernah kita buat tabelnya di *database*. Fungsi pencarian sangat penting dalam suatu program, karena mudah untuk melakukan pencarian data berdasarkan kebutuhan.

A. Membuat Kode Pencarian

Langkah pertama kita harus membuat *route* 'karyawan/cari' terlebih dahulu pada *file web.php*, berikut kode programnya.

```
Route::get('karyawan/cari',[KaryawanController::class, 'cari']);
```

Dan berikut kode program lengkapnya pada *file web.php*

```
<?php

use Illuminate\Support\Facades\Route;

use App\Http\Controllers\KaryawanController;

/*
|-----
|_____
```

```
| Web Routes
|-----
|
| Here is where you can register web routes for
| your application. These
| routes are loaded by the RouteServiceProvider
| within a group which
| contains the "web" middleware group. Now creat
| e something great!
|
*/

Route::get('/', function () {
    return view('welcome');
});

Route::get('karyawan', [KaryawanController::class
, 'index']);
Route::get('karyawan/tambah', [KaryawanController
::class, 'tambah']);
Route::post('/karyawan/store', [KaryawanControlle
r::class, 'store']);
Route::get('/karyawan/edit/{id}', [KaryawanContr
oller::class, 'edit']);
Route::post('/karyawan/update', [KaryawanControl
ler::class, 'update']);
Route::get('/karyawan/hapus/{id}', [KaryawanCont
roller::class, 'hapus']);
Route::get('laporan', [KaryawanController::class,
'laporan']);
Route::get('karyawan/cari', [KaryawanController::
class, 'cari']);
```

Karena di sini kita membuat method baru yaitu cari maka, langkah berikutnya adalah membuat method cari pada KaryawanController.php berikut kode programnya.

```
public function cari(Request $request)
```




```

{
    // menangkap data pencarian
    $cari = $request->cari;

    // mengambil data dari table karyawan sesuai pencarian data
    $pegawai = DB::table('karyawan')
        ->where('nama', 'like', "%".$cari."%")
        ->paginate();

    // mengirim data karyawan ke view index

    return view('index', ['karyawan' => $pegawai]);
}

```

Dan berikut kode lengkapnya pada *file* **KaryawanController.php**

```

<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB; //penggunaan query builder

class KaryawanController extends Controller

```



```

{
//halaman index di view
public function index()
{
    //fungsi pagination
    $karyawan = DB::table('karyawan')->
    >paginate(5);
    //fungsi mengambil data dari tabel karyawan
    // $karyawan = DB::table('karyawan')->get();

    //mengirim data karyawan ke bagian view
    return view('index', ['karyawan'=>$karyawan
    ]);
}

//halaman menampilkan tambah di view
public function tambah()
{
    return view('tambah');
}

//methode untuk insert data ke tabel karyawan
public function store(Request $request)
{
    //insert data ke tabel karyawan
    DB::table('karyawan')->insert([
        'nama' => $request->nama,
        'jabatan' => $request->jabatan,
        'umur' => $request->umur,
        'alamat' => $request->alamat
    ]);
    //kembali ke halaman karyawan
    return redirect('/karyawan');
}

//method untuk edit data karyawan
public function edit($id)
{
    //mengambil data berdasarkan id yang dipilih
    $karyawan = DB::table('karyawan')->
    >where('karyawan_id',$id)->get();

```



```

//passing data karyawan di dapat ke view edit
.balade.php
return view('edit', ['karyawan' => $karyawan])
;
}

//update data karyawan
public function update(Request $request)
{
    //update data pegawai
    DB::table('karyawan')-
    >where('karyawan_id', $request->id)->update([
        'nama' => $request->nama,
        'jabatan' => $request->jabatan,
        'umur' => $request->umur,
        'alamat' => $request->alamat
    ]);

    //kembali ke halaman karyawan
    return redirect('/karyawan');
}

//method untuk hapus data karyawan
public function hapus($id)
{
    //menghapus data karyawan berdasarkan id yang
    di pilih
    DB::table('karyawan')-
    >where('karyawan_id', $id)->delete();

    //kembali ke halaman karyawan
    return redirect('/karyawan');
}

public function laporan()
{
    //fungsi mengambil data dari tabel karyawan
    $karyawan = DB::table('karyawan')->get();

    //mengirim data karyawan ke bagian view
    return view('laporan', ['karyawan'=>$karyawan
    ]);
}

```



```

}

public function cari(Request $request)
{
    // menangkap data pencarian
    $cari = $request->cari;

    // mengambil data dari table karyawan sesuai
    pencarian data
    $pegawai = DB::table('karyawan')
    ->where('nama','like','%'.$cari.'%')
    ->paginate();

    // mengirim data karyawan ke view index
    return view('index',['karyawan' => $pegawai])
    ;

}
}

```

Setelah membuat method **cari** maka Langkah berikutnya adalah menambahkan *form* pencarian di **index.blade.php** berikut kode programnya.

```

<form action="/karyawan/cari" method="GET">

<input type="text" name="cari" placeholder="C
ari Nama Karyawan .." value="{{ old('cari') }}
">
    <input type="submit" value="CARI">
</form>

```

Untuk kode lengkapnya bisa dilihat d bawah ini.

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-
width, initial-scale=1.0">

```



```

<meta http-equiv="X-UA-
Compatible" content="ie=edge">

<!-- css tabel -->
<style>
table {
    border-collapse: collapse;
    }
    {
    width: 100%;
    }
    th
    { height: 50px; }

    b {
    background-color:#228B22; /* warna */
    border: none;
    color: white;
    padding: 15px 32px;
    text-align: center;
    text-decoration: none;
    display: inline-block;
    font-size: 16px;
    box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px
x 20px 0 rgba(0,0,0,0.19);
    }

    c {
    background-color:#FF8C00; /* warna */
    border: none;
    color: white;
    padding: 15px 32px;
    text-align: center;
    text-decoration: none;
    display: inline-block;
    font-size: 16px;
    box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px
x 20px 0 rgba(0,0,0,0.19);
    }

```



```

d {
background-color:#B22222; /* warna */
border: none;
color: white;
padding: 15px 32px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 16px;
box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6p
x 20px 0 rgba(0,0,0,0.19);
}

</style>
<title>Toko Sederhana</title>
</head>
<body align="center">

<table align="center" width=40%>
<tr>

<td><h2>Data Karyawan Toko Sederhana</h2><
/td>
</tr>

<tr>

<td><a href="/karyawan/tambah"><b align="c
enter">+Tambah Data</b></a>|<a href="/lapo
ran" target="_blank"><b align="center">Cet
ak Laporan</b></a></td>

</tr>
</table>
<br>

<form action="/karyawan/cari" method="GET"
>

<input type="text" name="cari" placeholder

```



```

        er="Cari Nama Karyawan .." value="{{ old
        ('cari') }}">
        <input type="submit" value="CARI">
    </form>
<br>

<table border=1 align="center">
<tr>
    <th>Nama</th>
    <th>Jabatan</th>
    <th>Umur</th>
    <th>Alamat</th>
    <th>Ops</th>
</tr>
@foreach($karyawan as $p)
<tr>
    <td>{{ $p->nama }}</td>
    <td>{{ $p->jabatan }}</td>
    <td>{{ $p->umur }}</td>
    <td>{{ $p->alamat }}</td>
    <td>
        <a href="/karyawan/edit/{{ $p-
        >karyawan_id }}"><c>Edit</c></a>
        |
        <a href="/karyawan/hapus/{{ $p-
        >karyawan_id }}"><d>Hapus</d></a>
    </td>
</tr>
</foreach>

</table>
<br/>
<p>Halaman : {{ $karyawan->currentPage() }}
| Jumlah Data : {{ $karyawan-
>total() }} | Data Per Halaman : {{ $karyawan
->perPage() }}</p>
    {{ $karyawan->links() }}
</body>
</html>

```



Jika sudah kita lihat hasilnya pada *link* berikut ini
<http://127.0.0.1:8000/karyawan>

Data Karyawan Toko Sederhana

+Tambah Data

Cetak Laporan

Cari Nama Karyawan ..

CARI

Nama	Jabatan	Umur	Alamat	Opsi	
Raja Sabaruddin	Direktur	25	Pontianak-Indonesia	Edit	Hapus
Pak Rajeee	Wakil Direktur	20	Kubu Raya- Indonesia	Edit	Hapus
bang raje	Direktur	21	Pontianak-Indonesia	Edit	Hapus
Sri Murni	Sekretaris	22	Subi-Indonesia	Edit	Hapus
Qona'ah	Karyawan Kasir	21	Subi-Indonesia	Edit	Hapus

Halaman : 1 | Jumlah Data : 7 | Data Per Halaman : 5

« Previous [Next](#) »



BAB 17



PENOMORAN FIELD OTOMATIS DI LARAVEL

Pada *project* sebelumnya kita belum ada menambahkan penomoran *filed* otomatis pada tampilan **index.php**. caranya tidak begitu sulit cukup menambahkan dua baris kode seperti berikut ini.

```
<?php $no=1;?>
```

Dan

```
<?php $no++ ;?>
```

Berikut kode lengkapnya pada *file* **index.blade.php**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-
width, initial-scale=1.0">
  <meta http-equiv="X-UA-
Compatible" content="ie=edge">

  <!-- css tabel -->
  <style>
    table {
      border-collapse: collapse;
```



```

}
{
width: 100%;
}
th
{ height: 50px; }

b {
background-color:#228B22; /* warna */
border: none;
color: white;
padding: 15px 32px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 16px;
box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px 20
px 0 rgba(0,0,0,0.19);
}

c {
background-color:#FF8C00; /* warna */
border: none;
color: white;
padding: 15px 32px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 16px;
box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px 20
px 0 rgba(0,0,0,0.19);
}

d {
background-color:#B22222; /* warna */
border: none;
color: white;
padding: 15px 32px;
text-align: center;
text-decoration: none;

```



```

        display: inline-block;
        font-size: 16px;
        box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px 20
px 0 rgba(0,0,0,0.19);
    }

    e {
        background-color:#D2691E; /* warna */
        border: none;
        color: white;
        padding: 15px 32px;
        text-align: center;
        text-decoration: none;
        display: inline-block;
        font-size: 16px;
        box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px 20
px 0 rgba(0,0,0,0.19);
    }

</style>
<title>Toko Sederhana</title>
</head>
<body align="center">

    <table align="center" width=40%>
    <tr>
    <td><h2>Data Karyawan Toko Sederhana</h2></td>
    </tr>

    <tr>

    <td><a href="/karyawan/tambah"><b align="center
">+Tambah Data</b></a>|<a href="/laporan" targe
t="_blank"><e align="center">Cetak Laporan</e><
/a></td>

    </tr>
    </table>
    <br>

```



```

<form action="/karyawan/cari" method="GET">

<input type="text" name="cari" placeholder="Car
i Nama Karyawan .." value="{{ old('cari') }}">
<input type="submit" value="CARI">
</form>
<br>

<table border=1 align="center">
<tr>
<th>#</th>
<th>Nama</th>
<th>Jabatan</th>
<th>Umur</th>
<th>Alamat</th>
<th>Ops</th>
</tr>
<?php $no=1;?>
@foreach($karyawan as $p)
<tr>
<td>{{ $no }}</td>
<td align="left">{{ $p->nama }}</td>
<td align="left">{{ $p->jabatan }}</td>
<td align="center">{{ $p->umur }}</td>
<td align="left">{{ $p->alamat }}</td>
<td>
<a href="/karyawan/edit/{{ $p-
>karyawan_id }}"><c>Edit</c></a>
|
<a href="/karyawan/hapus/{{ $p-
>karyawan_id }}"><d>Hapus</d></a>
</td>
</tr>
<?php $no++ ;?>
@endforeach

</table>
<br>
<p>Halaman : {{ $karyawan->currentPage() }}
| Jumlah Data : {{ $karyawan-
>total() }} | Data Per Halaman : {{ $karyawan-
>perPage() }}</p>

```



```
<p>{{ $karyawan->links() }}</p>
</body>
</html>
```

Sedangkan untuk laporan.blade.php berikut kode programnya.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-
width, initial-scale=1.0">
  <meta http-equiv="X-UA-
Compatible" content="ie=edge">

  <!-- css tabel -->
  <style>
    table {
      border-collapse: collapse;
    }
    {
      width: 100%;
    }
    th
    { height: 50px; }

    b {
      background-color:#228B22; /* warna */
      border: none;
      color: white;
      padding: 15px 32px;
      text-align: center;
      text-decoration: none;
      display: inline-block;
      font-size: 16px;
      box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px 2
0px 0 rgba(0,0,0,0.19);
    }

    c {
```



```

        background-color:#FF8C00; /* warna */
        border: none;
        color: white;
        padding: 15px 32px;
        text-align: center;
        text-decoration: none;
        display: inline-block;
        font-size: 16px;
        box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px 2
0px 0 rgba(0,0,0,0.19);
    }

    d {
        background-color:#B22222; /* warna */
        border: none;
        color: white;
        padding: 15px 32px;
        text-align: center;
        text-decoration: none;
        display: inline-block;
        font-size: 16px;
        box-
shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px 20
px 0 rgba(0,0,0,0.19);
    }

</style>
<title>Toko Sederhana</title>
</head>
<body align="center">

    <table align="center" width=40%>
    <tr>
    <td><h2>Data Karyawan Toko Sederhana</h2></td>
    </tr>
    <tr>

    <td>Jl Natuna, Kawasan Subi, Indonesia | No AAA
    8 | HP : 2876 78567 8756
    <hr></td>
    </tr>

```



```

        <tr>

</table>

<br>

<table border=1 align="center">
<tr>
    <th>#</th>
    <th>Nama</th>
    <th>Jabatan</th>
    <th>Umur</th>
    <th>Alamat</th>
</tr>
<?php $no=1;?>
@foreach($karyawan as $p)
<tr>
    <td>{{ $no }}</td>
    <td align="left">{{ $p->nama }}</td>
    <td align="left">{{ $p->jabatan }}</td>
    <td align="center">{{ $p->umur }}</td>
    <td align="left">{{ $p->alamat }}</td>

</tr>
<?php $no++ ;?>
@endforeach

</table>
<p>Dicetak di Pontianak, <?php echo date('l, d-
m-Y H:i:s'); ?></p>
</body>
</html>

```



Berikut hasilnya.

Data Karyawan Toko Sederhana

+Tambah Data

Cetak Laporan

Cari Nama Karyawan ..

CARI

#	Nama	Jabatan	Umur	Alamat	Opsi	
1	Raja Sabaruddin	Direktur	25	Pontianak-Indonesia	Edit	Hapus
2	Pak Rajeee	Wakil Direktur	20	Kubu Raya- Indonesia	Edit	Hapus
3	bang raje	Direktur	21	Pontianak-Indonesia	Edit	Hapus
4	Sri Murni	Sekretaris	22	Subi-Indonesia	Edit	Hapus
5	Qona'ah	Karyawan Kasir	21	Subi-Indonesia	Edit	Hapus

Halaman : 1 | Jumlah Data : 12 | Data Per Halaman : 5

« Previous [Next »](#)

Data Karyawan Toko Sederhana

Jl Natuna, Kawasan Subi, Indonesia | No AAA 8 | HP : 2876 78567 8756

#	Nama	Jabatan	Umur	Alamat
1	Raja Sabaruddin	Direktur	25	Pontianak-Indonesia
2	Pak Rajeee	Wakil Direktur	20	Kubu Raya- Indonesia
3	bang raje	Direktur	21	Pontianak-Indonesia
4	Sri Murni	Sekretaris	22	Subi-Indonesia
5	Qona'ah	Karyawan Kasir	21	Subi-Indonesia
6	Anggelika	Resepsionis	15	Subi-Indonesia
7	Sri Murni	Sekretaris	20	Boyan-Kabupaten Sanggau, Indonesia
8	Hasanah	Karyawan Kasir	23	Subi, Indonesia
9	Anggelika	Karyawan Kasir	21	SUbi, Indonesia
10	Istiqomah	Resepsionis	21	Kubu Raya, Indonesia
11	Miko	Satpam	27	Sungai Jawi, Kubu Raya, Indonesia
12	Reza	Resepsionis	34	Pontianak, Indonesia

Dicetak di Pontianak, Monday, 17-10-2022 10:43:24





Daftar Pustaka

- [1] Asmawi, Syafei, & Yamin, M. (2019). *Pendidikan Berbasis Teknologi Informasi Dan Komunikasi*.
- [2] Riyadli, H., Arliyana, & Saputra, F. E. (2020). Rancang Bangun Sistem Informasi Keuangan Berbasis Web. In *IJ Jurnal Sains Komputer dan Teknologi Informasi e-issn* (Vol. 3, Issue 1).
- [3] Anna, A., Nurmalasari, N., & Yusnita, A. E. (2018). Rancang Bangun Sistem Informasi Akuntansi Penerimaan dan Pengeluaran Kas pada Kantor Camat Pontianak Timur. *Jurnal Khatulistiwa Informatika*, 6(2), 107–118.
<https://doi.org/10.31294/khatulistiwa.v6i2.153>
- [4] Fadhallah, Dr. R. A. (2020). wawancara.
https://books.google.co.id/books?id=rN4fEAAAQBAJ&pg=PP4&ots=yxKJB3_86U&dq=wawancara%20adalah&lr&hl=id&pg=PP1#v=onepage&q=wawancara%20adalah&f=false
- [5] Supriyanta, & Agustiani, A. D. (2020). Perancangan Sistem Informasi E-Recruitment Karyawan Studi Kasus PT. Wahana Kasih Mulia Kedungreja. <http://apmmi.org>
- [6] Rosa, & Shalahuddin. (2018). *Rekayasa Perangkat Lunak Terstruktur dan Berorientasi Objek (revisi)*. Informatika Bandung.
- [7] Siddik, M. (2020). Rancang Bangun Sistem Informasi POS (Point Of Sale) Untuk Kasir Menggunakan Konsep Bahasa Pemrograman Orientasi Objek. *JOISIE Journal Of Information System And Informatics Engineering*, 4(1), 43–48.



- [8] Maydianto, & Ridho, M. R. (2021). Rancang Bangun Sistem Informasi Point of Sale dengan *Framework* Codeigniter pada CV Powershop. In *JURNAL COMASIE*.
- [9] Lestari, K. C., & Amri, A. M. (2020). Sistem Informasi Akuntansi Beserta Contoh Penerapan Aplikasi SIA Sederhana dalam UMKM. CV Budi Utama. <https://books.google.co.id/books?id=ShrWDwAAQBAJ&lpg=PP1&hl=id&pg=PA7#v=onepage&q&f=false>
- [10] Sulistiowati, Wibowo, J., & Pratama Putra, R. (2021). *Rancang Bangun Aplikasi Laporan Keuangan Berbasis Web*. Pada PT Anugrah Putra Kharisma.





Indeks

App, 13, 30, 34, 35, 39, 40, 43, 45,
48, 52, 63, 64, 65, 73, 74, 77,
81, 87, 91, 95, 100, 108, 109,
124, 125, 139, 144, 148, 155,
157

Bootstrap, 14

CMD, 12

Composer, 9

Config, 14

Database, vii, viii, 14, 55, 105,
106, 107, 114, 118, 123

htdocs, 10, 11, 12, 15

Laravel, iv, v, vi, vii, viii, ix, x, 1, 2,
4, 6, 7, 9, 10, 11, 12, 13, 14, 15,
17, 18, 19, 20, 25, 29, 30, 31,
33, 34, 36, 39, 41, 42, 43, 44,

47, 49, 50, 51, 53, 55, 56, 57,
58, 59, 61, 63, 65, 67, 68, 73,
74, 75, 76, 79, 80, 82, 83, 85,
87, 88, 95, 97, 105, 106, 107,
108, 134, 135, 147, 155, 165

project, 1, 9, 11, 12, 17, 18, 31, 33,
49, 53, 55, 58, 61, 65, 71, 95

Project, 1

Public, 14

Resource, 3, 14

Routes, 14, 35, 40, 45, 64, 91,
100, 125, 144, 156

Storage, 14

Test, 14

Vendor, 14







Profil Penulis

Penulis 1



Raja Sabaruddin M.Kom., seorang ahli komputer berprestasi, lulus S2 Ilmu Komputer, dan saat ini menjadi dosen di Universitas BSI Kota Pontianak. Saya tidak hanya memiliki gelar tinggi, tetapi juga semangat untuk mengeksplorasi pengetahuan yang memperkaya. Selain mengajar, saya aktif membentuk program inovatif yang mendorong semangat kewirausahaan mahasiswa. Keanggotaan saya di berbagai organisasi mencerminkan keterlibatan saya dalam dinamika komunitas. Sebagai penulis dengan buku ber-ISBN, saya berbagi pengetahuan di bidang komputer dan sistem informasi akuntansi. Proyek kolaboratif sukses saya adalah membangun sistem informasi rumah nelayan, memberikan dampak positif pada komunitas. Dengan semangat yang membara, saya siap terus berkontribusi dalam pendidikan dan penelitian, menantikan perjalanan panjang ke depan.



Penulis 2



Sri Murni M.Kom., lahir di Meliau tepatnya di Kampung Baru, tahun 1988. Kuliah berawal dari jenjang Diploma 3 lulusan dari Kampus Bina Sarana Informatika (BSI) Pontianak. Sebelum kuliah beliau sempat kursus 1 tahun di Data Computer Indonesia (DCI) mengambil Jurusan Komputerisasi Akuntansi. Bidang ini memang beliau minati dan beliau memiliki *background* pendidikan Sekolah Menengah jurusan akuntansi. Pada tahun 2012 menerima beasiswa untuk jenjang strata 1 dan lulus sebagai sarjana komputer. Tahun 2016 mendapat beasiswa kembali dan berhasil menyelesaikan program magister ilmu komputer dari STMIK Nusa Mandiri Jakarta. Pada saat masih menjadi mahasiswa jenjang diploma 3, beliau sudah terjun menjadi pengajar yang berawal dari menjadi asisten dosen dan ketika lulus beliau menjadi staf pengajar di Fakultas Teknologi Universitas Bina Sarana Informatika Kota Pontianak.

Penulis 3



Lisnawanty, S.T., M.Kom., Lulus S1 (S.T.) pada Program Studi Teknik Informatika Universitas Tanjungpura pada tahun 2009, dan lulus S2 (M.Kom.) pada Program Studi Ilmu Komputer STMIK Nusa Mandiri pada tahun 2012. Saat ini berperan sebagai Dosen Tetap pada Program Studi Informatika Universitas Bina Sarana Informatika dan Asesor Kompetensi pada LSP Universitas BSI. Telah kompeten pada bidang Pengembangan Perangkat Lunak (*Software Development*) dengan kualifikasi/kompetensi bidang



Pemrogram (*Programmer*) dari Badan Sertifikasi Nasional Profesi (BNSP). Pernah mengikuti beberapa pelatihan yang diselenggarakan oleh KOMINFO dalam Program Thematic Acedemy Digital Talent Scholarship, di antaranya Pelatihan *Data Scientist Artificial Intelligence* (2021), Pemanfaatan AI Chatbot (2022), dan *Women In Tech: Python and Cybersecurity* (2023). Aktif dalam publikasi ilmiah sejak tahun 2011 sampai dengan sekarang, baik tingkat nasional maupun internasional. Beberapa buku yang pernah diterbitkan, yaitu (1) Sistem Informasi Akuntansi: Teori dan Desain (terbit pada tahun 2019), dan (2) Pemrograman Akuntansi (terbit pada tahun 2022). Pernah menjadi narasumber dalam kegiatan Literasi Digital yang diselenggarakan oleh Kementerian KOMINFO Republik Indonesia, dan moderator dalam kegiatan yang diselenggarakan oleh Dinas KOMINFO Provinsi Kalimantan Barat. Serta ikut serta dalam beberapa kepengurusan organisasi dan keanggotaan profesi, di antaranya Asosiasi Pendidikan Tinggi Informatika dan Komputer (APTIKOM) Provinsi Kalimantan Barat sebagai pengurus, DPD Gradasi Provinsi Kalimantan Barat sebagai pengurus, Asosiasi Pendidikan Tinggi Informatika dan Komputer (APTIKOM) sebagai anggota, Asosiasi Dosen Indonesia (ADI) sebagai anggota, dan Persatuan Insinyur Indonesia (PII) sebagai anggota.



Penulis 4



Wahyu Nugraha, M.Kom., lahir di kota eksotis Pontianak pada 1 Maret 1987. Ia merajut kisah pendidikan gemilang, menyelesaikan S1 Sistem Informasi dan S2 Ilmu Komputer di STMIK Nusa Mandiri Jakarta. Kini, Wahyu tak hanya menaburkan ilmu sebagai dosen berdedikasi di Universitas BSI Kota Pontianak, tetapi juga mengisi lembaran hidup sebagai penulis berbakat. Dengan pena yang bersemangat, ia meracik pengetahuan dan pengalaman menjadi karya-karya yang menginspirasi. Dalam setiap kelas dan halaman tulisan, Wahyu mempersembahkan cerita pendidikan yang tak hanya informatif, tapi juga memikat hati.

